

סיכום לקורס מערכות זמן אמת ותכנות מקבילי

ע"פ השיעורים של ד"ר אריה טייטלבוים תשפ"א 2020-21

הכותב לא לוקח שום אחריות על טעויות מכל סוג בסיכום זה. הקריאה באחריות הקורא בלבד!

מצגת 1: Introduction

למערכת זמן אמת יש הגדרות שונות, אבל מה שמשותף לכולן הוא זמן התגובה של המערכת. ע"פ **Oxford Dictionary**: מערכת שבה הזמן להפיק פלט הוא משמעותי. בד"כ מערכות כאלה יהיו קשורות לתהליכים פיזיים ולכן התגובה חייבת להיות בהקשר לתנועה מסוימת באופן מיידי. הזמן בין קבלת הקלט להוצאת הפלט ידוע ומוגדר מראש.

ע"פ **Young**: כל פעילות או מערכת לעיבוד מידע שצריכה להגיב לגירויי קלט שנוצרו ממקור חיצוני בתוך פרק זמן מוגדר.

ע"פ **Randell**: מערכת הנדרשת להגיב לגירויים מהסביבה בפרקי זמן המוכתבים על ידי הסביבה. לכל מערכת יש את הזמנים שלה. הזמנים יכולים להיות ברמה דקות, שניות או מילי שניות אבל לכל מערכת יש הגדרות זמן (Timeliness).

נובע מכך שנכונות המערכת לא תלויה רק בנכונות הלוגיקה של המערכת אלא גם בזמן ביצוע אותה לוגיקה.

ניתן לחלק את מערכות ה-RTOS (Real Time OS) בפרט ומערכות בכלל לכמה סוגים:

Hard RTOS – התגובה של המערכת חייבת תמיד להיות בזמן המוגדר מראש. חריגה מהזמן היא אסון (לדוג' מערכת הגנה מפני טילים).

Soft RTOS – התגובה בזמן המוקצב היא חשובה אבל המערכת תדע להתמודד עם פספוסים שיקרו מידי פעם (לדוג' מערכת הזרמת וידאו למסך).

מערכת אינטראקטיבית – בשונה מ-Soft RTOS, מערכת אינטראקטיבית היא מערכת שלא מוגדרים בה הגדרות זמן מפורשות לביצוע משימות (לדוג' זמן התגובה של מקלדת).

Firm RTOS – נמצא בין Hard ל-Soft. ניתן לפספס את הדדליין אבל אין מה לעשות עם הפלט במקרה שהדדליין פוספס (לדוג' רובוט לניקוי הרצפה שמפספס אזור).

קיימות הגדרות חדשות יותר (קצת שונות מהקודמות) של Laplante:

Soft RTOS – מערכת בה הביצועים נחשבים פחות טובים אך לא נהרסים בגלל אי עמידה במגבלות זמן התגובה. אפילו תוכנות של עיבוד תמלילים (כמו Word) הן מערכות שצריכות להגיב בזמן סביר, אחרת הן יהיו לא נוחות לשימוש. הגדרה זו סותרת את ההגדרה שקדמה ל-Laplante.

Hard RTOS – מערכת עם דדליין קשיח שבה ניתן לסבול מספר קטן באופן שרירותי של דדליין שהוחמצו. מעט דדליינס שהוחמצו לא יובילו לכישלון מוחלט, אך מספר כישלונות גדול עלול להוביל לכשל שלם או קטסטרופלי של המערכת.

מערכות משובצות מחשב (Embedded Systems) הן מערכות שבהן המחשב מתממשק בדרך כלל ישירות לצידוד פיזי כלשהו והוא מוקדש לניטור או בקרה על תפעולו של אותו צידוד. כלומר המעבד הוא לא יחידה שנפרדת שמתחברת למערכת אלא היא רכיב שמשובץ בתוך המערכת.

הייתרון של מחשבים כאלה הוא שהם ייעודיים לתפקיד שלהם ואין להם שום הפרעות מגורמים אחרים ולכן ניתן להתאים אותו כך שהוא יעמוד בזמנים.

הגדרה נוספת (של Laplante): מערכת המכילה מחשב (או מעבד) אחד או יותר שיש לו תפקיד מרכזי בפונקציונליות של המערכת, אך המערכת אינה נקראת בפירוש מחשב.

פסיקה (Interrupt) היא הפסקה של התכנית המתבצעת כעת, על מנת לקרוא לפונקציה לטיפול בפסיקה (Interrupt Handler). בסיום הטיפול בפסיקה, התוכנית המקורית תחזור לרוץ, מהמקום שהופסקה. בעת ביצוע הפסיקה נשמרת כתובת החזרה במחסנית. פסיקה יכולה להיגרם ע"י חומרה או תוכנה.

פסיקת חומרה (Hardware Interrupt) מגיעה מההתקנים ההיקפיים, דרך בקר הפסיקות המסדרת אותם לפי תור עדיפויות, ומשם אל המעבד (לדוגמא לחיצה על מקש במקלדת).

פסיקות תוכנה (Trap) נגרמות ע"י תכנית ע"י ביצוע הפקודה המתאימה.

בעוד פסיקת תוכנה היא מרצון של התוכנית שרצה, פסיקת חומרה היא לא מרצון.

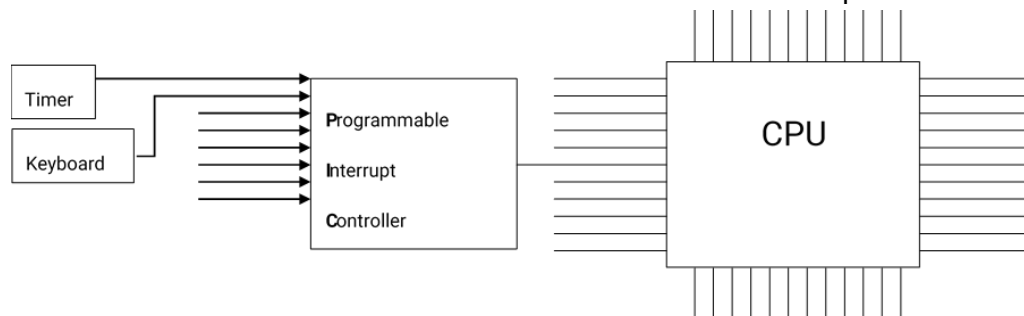
מערכת ההפעלה היא מונחת פסיקות (Interrupt Driven). כל פנייה אל מערכת ההפעלה נעשית באמצעות Trap, ורק כך מערכת ההפעלה יכולה לעבור ממצב User למצב Monitor וכך היא יכולה לבצע פעולות מועדפות. דבר זה מאפשר מנגנון הגנה.

הפסיקות מזוהות לפי מספר ולכן יש טבלה של כתובות של פונקציות לטיפול בפסיקה המכונה **וקטור הפסיקות (Interrupt Vector)** שהוא למעשה וקטור של מצביעים לפונקציות. שורה i בטבלה, היא הכתובת של הפונקציה המטפלת בפסיקה מספר i.

בעת ביצוע הפסיקה מתבצעות הפעולות הבאות:

1. שמירת כתובת החזרה במחסנית
2. גישה ל Interrupt Vector
3. קריאת Interrupt Handler המתאים
4. שמירת ערכי האוגרים במחסנית
5. טיפול בפסיקה
6. סיום הטיפול בפסיקה
7. שליפת ערכי האוגרים מהמחסנית
8. שליפת כתובת חזרה מהמחסנית
9. חזרה לתוכנית

בקר הפסיקות (PIC או Interrupt Controller) הוא רכיב בעל 8 כניסות ויציאה אחת שתפקידו לנהל תור עדיפויות ולבחור איזה פסיקה להעביר למעבד.



בתוך הבקר יש 3 רגיסטרים:

IRR (Interrupt Request Register) הוא רגיסטר בעל 8 סיביות שמסמן איזה רגליים של הבקר קיבלו פסיקה (כלומר אם רגל מספר 0 קיבלה פסיקה אזי בIRR הסיבית 0 תהיה דלוקה).

ISR (In-Service Register) הוא רגיסטר בעל 8 סיביות שמסמן איזה פסיקות כרגע פעילות והעוברים לטיפול של הCPU.

IMR (Interrupt Mask Register) הוא רגיסטר מציין מאיזה פסיקות יש להתעלם ולא לאשר.

קינון פסיקות הוא מצב שבו הגיעה פסיקה בעלת עדיפות גבוהה יותר מהפסיקה שכרגע רצה. במקרה כזה ניתן לעצור את הפסיקה שרצה, להריץ את הפסיקה החדשה ולאחר מכן לחזור לפסיקה הראשונית.

בסיום טיפול בפסיקה (מצב המכונה **End Of Interrupt** או בקיצור **EOI**) הCPU מודיע לבקר שהטיפול הסתיים בעזרת bus שמקשר ביניהם.

Interrupt Function: ע"י הוספת המילה השמורה interrupt בC/C++ לפני הגדרת פונקציה, המהדר (compiler) מבין שמדובר בInterrupt Handler, ולכן בשונה מפונקציה רגילה הוא:

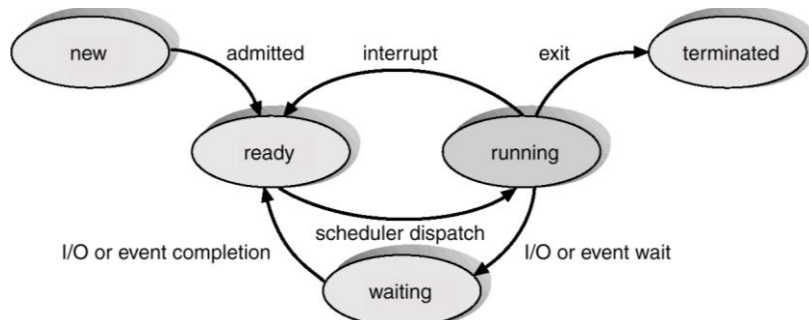
1. שותל פקודת Push עבור כל הרגיסטרים בתחילת הפונקציה.
2. שותל פקודת Pop עבור כל הרגיסטרים בסוף הפונקציה.
3. פקודת Ret מוחלפת בפקודת IRet.

בקריאה לטיפול בפסיקה נשמרים כל מיני Flags של המעבד שלא נשמרים במעבר רגיל בין פונקציה לפונקציה. לכן במצב כזה יש צורך להחליף את הביצוע של return רגיל (Ret) בכזה שיודע לשחזר את אותם דגלים (IRet).

Multi-threading: כשקיים רק מעבד אחד, וכמה תהליכים שאמורים לרוץ במקביל, מערכת ההפעלה מחלקת את זמן המעבד בין התהליכים (threads) לפי קריטריון מוסכם מראש.

על מנת לחלק את זמן המעבד בין התהליכים יש לנהל מבנה נתונים מתאים (לדוגמא PCB ממערכות הפעלה שנוצר עבור כל תהליך ושומר את הנתונים הדינאמיים על התהליך) בו ישמרו נתוני התהליכים, יש לקבוע מנגנון להפסקת הרצה של תהליך: Preemptive (כשיש חטיפה של המעבד בכוח) או Non Preemptive (כשאין חטיפה בכוח), יש לקבוע איזה נתונים צריכים לשמור/לשחזר וכן יש צורך בשעון שמודד זמן (לדוג' לחישוב הזמן בRR). במערכת Real Time יש חשיבות להחלפת המשימות שרצות על CPU גם אם משימה כלשהי לא סיימה לרוץ. לא ניתן לסמוך על המשימה עצמה שתפסיק את הריצה שלה ברגע שיש משימה בעלת עדיפות גבוהה יותר שנכנסת לתור Ready של המעבד, ולכן כל כמה זמן Timer שולח פסיקה שמבצעת בדיקה על מנת לברר לאיזה תהליך מגיע לרוץ.

תהליך יכול להיות באחד מהמצבים הבאים:



חשוב לשים לב שהמעבר למצב running מתבצע אך ורק דרך התור של ready, שם נקבע הסדר לפי הסדרן. קיימים אלגוריתמים (קריטריונים) שונים לחלוקת זמן המעבד אבל עבור מערכות Real-Time קיימים אלגוריתמים מיוחדים שהם מותאמים למגבלות מערכות אלו. לכל תהליך נשמור מבנה נתונים שיכיל נתונים כמו: שם, מצב, עדיפות (מקורית ועכשווית), ערך האוגרים, Program Counter, Stack Pointer ועוד.

RTOS לעומת OS רגילה תשים דגש רב יותר על הפיצ'רים הבאים:

1. טיפול מהיר בפסיקות
2. הגנה על זיכרון דרך החומרה ולא דרך התוכנה, כיוון שהחומרה מהירה יותר מהתוכנה
3. Preemption שבא לידי ביטוי בפסיקות
4. תקורה מינימלית של קריאות למערכת ההפעלה
5. רמות עדיפויות מרובות
6. תמיכה בPriority inheritance
7. גרעין (של הOS) שלא תופס הרבה זיכרון ומודולרי בהתאם ללקוח
8. סדרנים בעלי זמן תגובה צפוי מראש

בארץ קיימות בשוק RTOS כמו VxWorks, Integrity, Embedded Linux, FreeRTOS ועוד.

מצגת 2: SMARTS

SMARTS (Small Real-Time System) היא מערכת מרובת משימות (Multi-Threading) תחת DOS. מערכת זו מסוגלת להריץ כמה משימות במקביל ולהעביר מסרים ביניהם. המערכת כתובה בקוד פתוח ולכן ניתן לשנותה. על מנת שמערכת תהיה מסוגלת להריץ כמה משימות במקביל יש צורך באמצעי שדרכו ניתן להפסיק "בכוח" משימה באמצע ביצוע (Preemption), באפשרות לשחזור משימה שהופסקה (Resume) ובשעון למדידת יחידות זמן-שימוש המעבד (Timer).

במחשב קיים רכיב המהווה את Timer. רכיב זה שולח פסיקה שמספרה 8 לבקר הפסיקות כל כ-55 אלפיות השנייה. כאשר מגיע פסיקה כזאת מופעל Timer Interrupt Handler עליו מצביע וקטור הפסיקות.

BIOS (Basic Input-Output System) היא מערכת קושחה בסיסית שצורבה על ה-ROM שתפקידה בין היתר להעלות את מערכת ההפעלה מתוך הכונן הקשיח בעת הפעלת המחשב.

כאשר מתבצעת הדלקה של המחשב (boot) ה-BIOS שותל בווקטור הפסיקות פונקציה משלו עבור פסיקה מספר 8 (פסיקת Timer). לכן בעת עליית מערכת ההפעלה בווקטור הפסיקות בכניסה המתאימה לפסיקה מספר 8, יש כבר כתובת של Interrupt Handler.

מתפקידי BIOS Timer Interrupt Handler:

- קידום המונים של השעה והתאריך
- טיפול בכיבוי מנוע כונן הדיסקטים (שהיה נכבה אחרי 3 שניות שבהן לא היה קריאה מהדיסקט)
- שרשור לפסיקה מספר 0x1C (בעבר כשלא היה ריבוי תהליכים ה-BIOS הציע למשתמשים להשתיל Interrupt Handler בווקטור הפסיקות עבור פסיקה מספר 0x1C והוא היה קורא לפסיקה זו כל פעם בסיום הטיפול בפסיקה 8)

למעשה ה-Timer Interrupt מספק בפני עצמו שתיים מהדרישות של RTOS:

1. אמצעי שדרכו ניתן להפסיק בכוח משימה באמצע ביצוע (Preemption)

2. שעון למדידת יחידות זמן שימוש המעבד (Timer)

על מנת להפוך את SMARTS למרובת תהליכים יש צורך לדרוש את Handler של פסיקה 8 שמושתל ע"י ה-BIOS. מערכת SMARTS תעביר את Handler המקורי של ה-BIOS עבור פסיקה 8 למקום 60 בווקטור הפסיקות ולכן כשתבצע פסיקה 8 בפועל תופעל פונקציה של SMARTS, היא תפעיל פסיקת תוכנה מספר 60 שתקרא לפונקציה של ה-BIOS ובסיומה תחזור לרוץ הפונקציה של SMARTS (ע"י פקודת IRet).

לא ניתן לשתף מחסנית בין כל התהליכים. נניח שנתונים 2 התהליכים הבאים:

Thread A	Thread B
...	...
call a1();	call b1();
...	...

נניח ש-A התחיל לרוץ וקרא לפונקציה a1. לתוך המחסנית תיכנס כתובת החזרה ל-A, ולכן המחסנית תראה כך:

כתובת חזרה ל-A

לאחר מכן נניח שהיה Context Switch ותהליך B התחיל לרוץ וקרא לפונקציה b1. במקרה כזה תיכנס למחסנית כתובת החזרה ל-B ולכן המחסנית תראה כך:

כתובת חזרה ל-B
כתובת חזרה ל-A

לאחר מכן נניח שהיה Context Switch נוסף ו-A חזרה לעבוד. בסיום הריצה של a1 היא תחזור לכתובת האחרונה שנמצאת במחסנית משום שבפקודת return מתבצע שליפה של כתובת החזרה מהמחסנית אל ה-PC. במקרה כזה בסיום הפונקציה a1 התוכנית תריץ קוד של תהליך B במקום לחזור לקוד של תהליך A. מסקנה: יש לנהל מחסנית עבור כל תהליך בנפרד.

לכן בעת מעבר בין תהליך לתהליך צריך לשמור גם את Stack Pointer הספציפי לאותו תהליך. למעשה, הכתובת עליו מצביע Stack Pointer באמצע ביצוע פסיקה, מכיל את המידע שנדרש על מנת לשחזר את התהליך ממצב

של הקפאה. ולכן החלפת ערך Stack Pointer נוכחי בערך של Stack Pointer אחר, מתפרש באופן לוגי כContext Switch.

לפעמים יש צורך לקרוא לסדרן שנמצא בכתובת 8 בוווקטור הפסיקות באמצעות פסיקת תוכנה (לדוג' בסיום ריצה של תהליך). במצב כזה אין צורך לקרוא לפונקציה המקורית של BIOS שנמצאת בכתובת 60. לכן על המערכת להתייחס למקור הפסיקה, ולקרוא לפונקציית BIOS אך ורק כשמקור הפסיקה הוא חומרה. בנוסף, לפעמים צריך להבדיל גם עבור מקרה הפוך: אם המערכת באמצע הרצה של קטע אטומי יש צורך לחסום פסיקות תוכנה ולא לבצע Context Switch אבל כן יש צורך לזהות פסיקות חומרה ולהריץ את פונקציית BIOS. בSMARTS הקטע האטומי מומש ע"י flag גלובלי: כאשר ערך flag הוא true ניתן לבצע Context Switch. הפתרון הזה לא כ"כ אלגנטי, לדוג' במימוש של acquire במיוטקס: כל פונקציית acquire צריכה להיות אטומית אבל אם התהליך צריך להרדים את עצמו בתור הוא יצטרך להפעיל את פסיקה 8 בשביל לקרוא לתהליך הבא ונוצר מצב שבו צריך לפרק את acquiren לכמה קטעים קריטיים קטנים. בשביל לפתור את הבעיה הנ"ל ניתן להחליט שContext Switch יתבצע רק ע"י פסיקות חומרה. כלומר אם מתבצעת פסיקת תוכנה אין צורך להתייחס לflag של Context Switch כיוון שלא יכול להיות מצב שבו פונקציה קוראת לפסיקה 8 אבל עדיין תרצה להישאר בתוך קטע אטומי. כאשר עוברים לתהליך אחר אחרי Context Switch הוא לא אמור לרוץ כקטע אטומי אבל אם התהליך הקודם היה בעצמו בתוך קטע אטומי וקרא לפסיקת תוכנה, אזי גם התהליך הבא ירוץ כקטע אטומי ופסיקות חומרה ישאירו אותו כקטע אטומי. לכן יש צורך במימוש flag של קטע אטומי נפרד לכל תהליך. במערכת VxWorks הקטע האטומי מומש ע"י flag פרטני לכל תהליך וחסומה רק במקרה של פסיקת חומרה. בSMARTS יש דגל ייעודי proglnt שנדלק כאשר מתבצעת פסיקת תוכנה וכך ניתן להבדיל בין פסיקת חומרה לתוכנה.

סיום תהליך מתבצע ע"י: שינוי מצב התהליך ממצב Ready למצב NotActive וביצוע פסיקת תוכנה של Timer. לאחר מכן על הסדרן לבחור תהליך שנמצא במצב Ready להמשך הרצתו. התהליך שסיים, שוב לא ייבחר ע"י הסדרן. על מנת לחסוך מהמתכנת לבצע את הדברים האלו באופן ידני בתחתית המחסנית יש כתובת חזרה לTaskEnd, המבצעת את שינוי המצב ופסיקת התוכנה. בצורה כזאת בעת הרצת Ret בסיום התהליך המערכת תריץ למעשה את הפונקציה TaskEnd.

SMARTS מספק כלי לסנכרון והעברת מסרים ע"י שליחת אירועים (Event). אירועים אלו מיושמים ע"י מחלקות מסוג class Event. בשונה מהEvents של הקורס הקודם, בקורס הזה Events נשלחים לתהליך ספציפי ולא באופן כללי לתהליך אחר שמחכה בצד השני (כלומר הEvents הם מסוג direct). מחלקה זו מספקת שתי מתודות:

1. **Send**: לשליחת האירוע – send(char name, void *param, int synch)

2. **Wait**: לקבלת האירוע – wait(char &sourceP) // (sourceP will contain the task sender ID)

שליחת האירוע מכוונת ליעד (תהליך) מסוים, אך קבלת האירוע לא מכתובה מקור מסוים. אם התהליך המקבל מנסה לקבל אירוע, אך התהליך השולח טרם שלח, במקרה זה התהליך המקבל "נרדם" (נכנס למצב Suspend) עד מועד הגעת האירוע.

שליחת האירוע יכולה להיות: Synchronous או Asynchronous. במקרה של שליחה מסונכרנת, התהליך השולח והתהליך המקבל אמורים "להיפגש", ולכן במקרה והתהליך השולח שלח אירוע סינכרוני, והתהליך המקבל עדיין איננו מחכה לקבל את האירוע, התהליך השולח יירדם עד מועד קבלת האירוע.

מצגת 3: Real-Time Scheduling

הסדרן (Scheduler) הוא הלב של כל kernel, והוא מספק את האלגוריתם הדרוש כדי לקבוע איזו משימה מבצעת ומתי.

במערכות הפעלה שהן לא מערכות זמן אמת המטרה האופיינית של הסדרן היא למקסם את ה average throughput (התפוקה הממוצעת - מספר המשימות שהושלמו ליחידת זמן) ו/או למזער את זמן ההמתנה הממוצע של המשימות. כל זה נבע מכך שב OS אין דאגה לכל תהליך בנפרד אלא לכלל המערכת, והיחס לכל התהליכים הוא כשווי ערך פחות או יותר.

במערכות בזמן אמת לעומת זאת, המטרה היא לעמוד בדדליין של כל תהליך על ידי הקפדה שכל משימה תוכל להשלים את הביצוע שלה עד הסוף. במערכות RTOS יכול להיות שזמן ההמתנה בתור ready יהיה גבוה ויכול להיות שה average throughput יהיה נמוך אבל זה לא משנה כל עוד אף תהליך לא פספס את הדדליין שלו. הדגש הוא על תהליך בנפרד ולא על כל המערכת ככלל.

Schedulability Test היא נוסחה או אלגוריתם שמחושבת באופן ידני עוד לפני הרצת המערכת. משמש לאימות כי יישום נתון יכול לעמוד בדדליין של התהליכים בו כאשר הוא מתוזמן על פי אלגוריתם תזמון ספציפי. מבחן זה מאפשר בדיקה מקדימה על מנת לראות שהמערכת אכן מתאימה למה שהיא צריכה לבצע. במקרה והמערכת לא עברה את המבחן יש לשקול הוספת מעבדים, שינוי אלגוריתם הסדרן וכו'.

קיימים 3 משפחות של תהליכים:

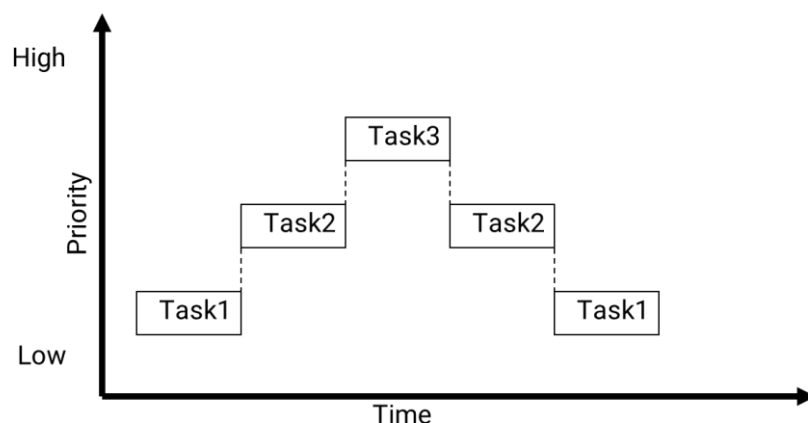
1. **Single instance**: כל תהליך מתבצע פעם אחת בלבד.
2. **Periodic**: כל תהליך מתבצע איטרציות רבות עם הפסקה קבועה ביניהן. כל תהליך חייב להתחיל לרוץ אחרי זמן תחילת המחזור ולסיים לא אחרי זמן סיום המחזור. תהליכים כאלו הם תהליכים שמתאימים במיוחד ל RTOS. דוגמא: מכ"ם שמחפש טיל בשמיים כל פרק זמן מסוים.
3. **Sporadic**: כל תהליך רץ אפס או יותר מקרים עם הפרדה מינימלית ביניהם. כלומר לא ידוע אם בכלל התהליך ירוץ, אבל אם הוא אכן ירוץ יש פרק זמן מסוים אותו יש להמתין בין הרצה להרצה. למשל הפעלת מטף חירום שברגע שמפעילים אותו יש השהיית של 5 שניות עד ההפעלה הבאה.

אלגוריתמים של סדרנים לתהליכים ממשפחת Single Instance:

(Fixed Priority Scheduling) FPS

הגישה הנפוצה ביותר לכל תהליך יש עדיפות קבועה, סטטית, שנקבעת ע"י המתכנת עוד לפני הריצה. אלגוריתם זה הוא fixed (סטטי ולא משתנה במהלך הריצה). התהליכים רצים לפי סדר העדיפות שלהם ואין התיישנות - תהליך נשאר באותה עדיפות לכל אורך החיים שלו.

גם האלגוריתם הזה הוא preemptive כיוון שכל Δt כלשהו (55 מילי שניות בד"כ) הסדרן נקרא כדי לבדוק האם לתת לתהליך לרוץ או שאולי יש תהליך בעל עדיפות גבוהה יותר שצריך לחטוף לתהליך שרץ את המעבד. ב RTOS העדיפות של משימה נגזרת מדרישות הזמן שלה, ולא מחשיבותה או מתפוקת המערכת שלה. דוגמת הרצה:

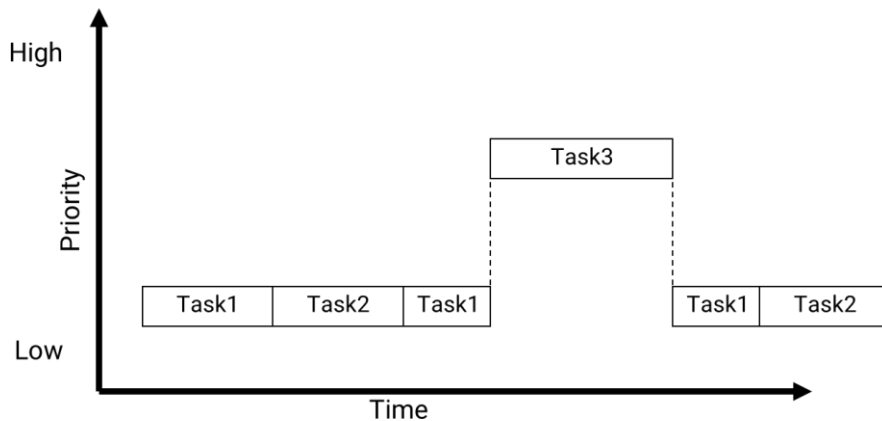


(Round Robin Scheduling) RR

אלגוריתם שמספק לכל תהליך חלק שווה (quantum או q) בזמן הריצה על המעבד, כך שכשלתהליך נגמר פלח הזמן הוא יאלץ לחכות $q \cdot (n-1)$ יחידות זמן לפני שירוצן שוב. אלגוריתם זה הוא fixed (סטטי ולא משתנה במהלך הריצה).

סדרן RR בפני עצמו אינו יכול לספק את הדרישות של RTOS אבל ניתן להשתמש בו לתזמון תהליכים בעלי אותה רמת עדיפות.

בVxWorks לדוגמא יש סדרן שמכיל מערך של תורים. כל כניסה במערך מייצגת רמת עדיפות מסוימת, כך שכל פעם שהסדרן מחפש איזה תהליך להריץ הוא מתחיל בכניסה בעלת העדיפות הגבוהה ביותר, מחפש את התור הראשון במערך שהוא לא ריק ומריץ את התהליך הראשון בתור. דוגמת הרצה:



אלגוריתמים של סדרנים לתהליכים ממשפחת Periodic:

(Rate Monotonic Scheduling) RMS

אלגוריתם תזמון preemptive, המשתמש בשיטת הקצאת העדיפות המונוטונית כדי להקצות לתהליך את העדיפות שלו. שיטה זו נותנת עדיפות גבוהה יותר לתהליך עם זמן מחזור קצר יותר מכיוון שהנחת היסוד שלה היא שמי שיש לו זמן מחזור קטן יותר הוא יותר לחוץ (הנחה זו נכונה ברוב המקרים אבל לא תמיד). למעשה אלגוריתם זה הוא סוג של FPS שבו העדיפות נקבעת ע"פ זמן המחזור ולכן אלגוריתם זה הוא fixed (סטטי ולא משתנה במהלך הריצה).

(Rate Monotonic Analysis) RMA הוא Schedulability Test של RMS. מטרת המבחן הוא לבדוק עוד לפני הריצה האם המערכת תעמוד בזמנים המוגדרים ע"י חישוב הניצול של המעבד ע"י התהליכים. בהנחה שמתקיימים התנאים הבאים:

- כל התהליכים הם מחזוריים
- אין אינטראקציה בין התהליכים
- המועד האחרון לסיום ביצוע של תהליך הוא תחילת זמן המחזור הבא שלו
- לכל תהליך יש זמן ריצה קבוע

אם התנאים הנ"ל מתקיים ניתן יהיה להשתמש בנוסחה של RMA:

$$\sum_{i=1}^n \frac{C_i}{T_i} \leq N(2^{1/N} - 1)$$

כאשר C הוא זמן הריצה, T הוא זמן המחזור וN מספר התהליכים שרצים. אם התנאי בנוסחה מתקיים אזי בהכרח כל התהליכים יעמדו בדדליין שלהם.

כיוון שRMS יוצא מנקודת הנחה שהיא לא תמיד נכונה, צריך לקחת בנוסחה מרווח ביטחון ולא ניתן להגיע עם אלגוריתם זה לניצול מלא של המעבד.

RMA היא בדיקה מספקת אך אינה הכרחית. כלומר אם קבוצת תהליכים עוברת את המבחן, היא תעמוד בכל הדדליין של כל התהליכים. אבל מצד שני אם הקבוצה נכשלת במבחן, זה לא בהכרח אומר שהיא תיכשל גם בזמן ריצה.

EDF (Earliest Deadline First)

באלגוריתם זה התהליכים רצים לפי הסדר שנקבע על פי המועדים המוחלטים של התהליכים, כלומר התהליך הבא שירוך הוא זה עם הדדליין הקרוב ביותר (המשימה שהכי לחוץ לסיים את ההרצה שלה). אלגוריתם זה משתמש בתזמון עדיפות **דינאמי** כיוון שהעדיפויות נקבעות בזמן ריצה. אלגוריתם זה מצליח לנצל קצת יותר טוב את המעבד לעומת RMS אבל הוא לא דטרמיניסטי כי העדיפויות כל הזמן משתנות בזמן הריצה.

EDF test הוא ה Schedulability Test של EDF. הנוסחה של EDF test מאוד דומה לנוסחה של RMA אבל בנוסחה זו כן ניתן להגיע לניצול מלא של המעבד והיא פשוטה יותר. הנוסחה של מבחן זה היא הנוסחה הבאה:

$$\sum_{i=1}^N \frac{C_i}{T_i} \leq 1$$

EDF תמיד יכול לתזמן כל קבוצת תהליכים שRMS יכול, אבל לא להפך. למרות כל זאת היתרונות של RMS הם שהוא קל יותר ליישום משום שיש לו עדיפות סטטית, פחות תקורה של חישובים, צפוי יותר בתנאים לא יציבים ומתאים גם כאשר לא מוגדר דדליין בכלל (כלומר הוא מסוגל לשלב גם תהליכים מסוג Single instance).

LLF (Least Laxity First)

יהי $c(i)$ זמן הריצה שנותר עבור התהליך בזמן i , ויהי $d(i)$ הזמן שנותר עד הדדליין של התהליך בזמן i . נגדיר את laxity (רפיון) של תהליך בזמן i להיות $l(i) = d(i) - c(i)$. יוצא מכך שהלaxity של תהליך הוא הזמן המרבי שניתן לעכב את הביצוע מבלי להחמיץ את הדדליין של התהליך. האלגוריתם ייבחר כל פעם את התהליך שהלaxity שלו הוא הכי קטן.

גם אלגוריתם זה הוא אלגוריתם דינאמי כי חישוב העדיפויות מתבצע תוך כדי הריצה. מסיבה זו האלגוריתם מבזבז המון זמן בחישובים שלוקחים לפעמים זמן יקר לעומת EDF שהוא קצת פשוט יותר. באלגוריתם זה יש גם חיסרון מרכזי כאשר יש מצב שקיימים 2 תהליכים בעלי laxity צמודים. נניח שנתונים 2 תהליכים T_1 T_2 בעלי laxity הבאים:

$$l_{T_1}(i) = d_{T_1}(i) - c_{T_1}(i) = 8$$

$$l_{T_2}(i) = d_{T_2}(i) - c_{T_2}(i) = 9$$

הסדרן יבחר ב T_1 כי laxity שלו הכי נמוך. בפעם הבאה שתהיה פסיקה והסדרן יחשב מחדש יתקבל:

$$l_{T_1}(i) = d_{T_1}(i) - c_{T_1}(i) = 8$$

$$l_{T_2}(i) = d_{T_2}(i) - c_{T_2}(i) = 8$$

הסדרן ייבחר שוב ב T_1 (בגלל הסדר שלהם) ובפעם הבאה הסדרן יקבל laxity הבאים:

$$l_{T_1}(i) = d_{T_1}(i) - c_{T_1}(i) = 8$$

$$l_{T_2}(i) = d_{T_2}(i) - c_{T_2}(i) = 7$$

במקרה הזה הסדרן יבחר ב T_2 ויבצע Context Switch. אחרי כמה איטרציות נוספות יקרה מצב הפוך שבו T_1 יהיה נמוך יותר וכך הלאה: כל פעם T_1 T_2 יתחלפו ביניהם ויבזבזו הרבה זמן על Context Switch מיותרים. EDF במצב כזה היה נותן לאחד מהם לרוץ עד הסוף ורק לאחר מכן היה נותן את המעבד לתהליך השני.

משפט ג'קסון (Theorem of Jackson) טוען שכל סדרן שמזמן את התהליך עם הדדליין המוקדם ביותר (כלומר EDF) בין כל התהליכים האחרים הוא אופטימלי ביחס להקטנת האיחורים של התהליכים. במילים אחרות: EDF הוא הסדרן הכי טוב.

מצגת 4: Real-Time Synchronization

Race Condition הוא מצב לא רצוי המתרחש כאשר מכשיר או מערכת מנסים לבצע שתי פעולות או יותר בו זמנית, אך בגלל אופי המכשיר או המערכת, הפעולות חייבות להיעשות ברצף המתאים כדי לפעול נכון. בהקשר של גישה לזיכרון, race condition עלול להתרחש אם פקודות לקריאה וכתבייה של נתונים מתקבלות כמעט באותו זמן, והמכונה מנסה להחליף חלק מהנתונים הישנים או את כולם, בעוד הנתונים הישנים עדיין נמצאים כמו שהם אצל הקורא.

קטע קריטי (Critical Section) הוא קטע קוד שניגש למשאב משותף (מבנה נתונים או מכשיר) שאסור לגשת אליו במקביל ביותר מתהליך אחד. כאשר תהליך אחד נמצא בקטע הקריטי, אסור לאפשר לאף תהליך אחר להיות באותו קטע קריטי. ניהול נכון של קטע קריטי מהווה פתרון ל race condition. פתרון לבעיית הקטע הקריטי חייב לעמוד בשלוש הדרישות הבאות:

1. מניעה הדדית
2. התקדמות
3. המתנה חסומה

ב RTOS התנאי של המתנה חסומה קצת משתנה, כיוון שזה לא הגיוני לתעדף משהו במעבד אבל לא לתעדף אותו בכניסה לקטע קריטי ושם לתקוע אותו. לכן בקטעים קריטיים ב RTOS התור לקטע קריטי יכול להיות ממין לפי FIFO או לפי עדיפויות.

Semaphores הוא כלי לסנכרון שמכיל 3 פונקציות:

1. Acquire או Wait בכניסה לסמפור
2. Release או Signal ביציאה מהסמפור
3. Flush שמשחרר את כל התהליכים שבתור אל תוך הקטע הקריטי

סמפור בינארי הוא סמפור שנותן גישה רק לתהליך אחד בו זמנית: 0 מסמן שהסמפור תפוס (ב VxWorks זה נקרא empty וזה עשוי להישמע מבלבל) או 1 שמסמן שהסמפור פנוי (ב VxWorks זה נקרא full). כל תהליך יכול לעשות acquire וכל תהליך יכול לעשות release, כלומר אין חובה שמי שתפס הוא זה שישחרר.

סמפור מניה הוא סמפור שמאפשר גישה לכמה תהליכים במקביל: 0 מסמן שאין מקום פנוי בסמפור, כל ערך שגדול מ 0 מסמן שניתן להיכנס לסמפור. גם פה כל תהליך יכול לעשות acquire וכל תהליך יכול לעשות release, כלומר אין חובה שמי שתפס הוא זה שישחרר.

מיוטקס (Mutex) הוא סמפור בינארי שתומך בבעלות, כלומר מעבר לשדות הרגילים של סמפור בינארי יש לו שדה ששומר את הבעלים של המיוטקס. דבר זה מאפשר שרק התהליך שעשה acquire הוא זה שיעשה release, גישה רקורסיבית ומניעת שליחה של פקודת kill לאותו תהליך (דבר שיכול לתקוע תהליכים אחרים בתור כי אם התהליך שהיה ה owner לא קיים יותר אין מי שיוציא אותם מהתור).

ב RTOS הצורך לחלוק משאבים בין משימות הפועלות בסביבת ריבוי משימות שהיא preemptive עלול ליצור בעיות. שתיים מהבעיות הנפוצות ביותר הן Priority Inversion ו Priority Inversion, שניהם עלולים לגרום לכישלון של המערכת.

Deadlock הוא מצב שמתרחש כששני תהליכים מחכים כל אחד שהשני יסתיים לפני שהם ממשיכים. התוצאה היא ששני התהליכים נתקעים.

Coffman Conditions הם תנאים שמגדירים מתי ניתן להגיע למצב של deadlock: deadlock על משאב יכול להיווצר אם"ם כל התנאים הבאים מתקיימים בו זמנית במערכת:

1. **Mutual Exclusion / מניעה הדדית** - לפחות משאב אחד לא ניתן לשיתוף. כלומר רק תהליך אחד יכול להשתמש במשאב בכל רגע נתון.
2. **Hold and Wait / מחזיק ממתין** - תהליך מחזיק לפחות משאב אחד ומבקש משאבים נוספים המוחזקים על ידי תהליכים אחרים.
3. **No Preemption / אין חטיפה** - לא ניתן לחטוף משאב מהתהליך שמחזיק אותו. משאב משוחרר רק מרצון של הבעלים.

4. Circular Wait / המתנה מעגלית - כל תהליך חייב להמתין למשאב המוחזק על ידי תהליך אחר כך

שהתהליך האחרון בשרשרת מחכה למשאב שתפוס ע"י התהליך הראשון בשרשרת.

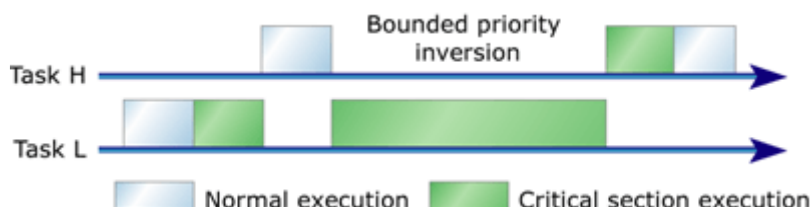
באופן כללי כדאי ניתן להימנע מdeadlock בדרכים הבאות:

- אם החלקים הקריטיים קצרים מאוד, ניתן להפוך אותם לאטומים. אם הקטע קריטי ארוך זה עלול לגרום לקיפוח.
- ביצוע acquire לסמפורים בסדר עולה (הפתרון של דייקסטרה בבעיית הפילוסופים הסועדים). רלוונטי רק לתהליכים שבהם ידוע מראש איזה משאבים התהליך הולך לבקש.
- לא לבצע acquire לסמפור נוסף, לפני שהתהליך שחרר את הקודם.
- ביצוע acquire לכל הסמפורים הדרושים ברצף בצורה אטומית, אחרי שמוודאים שכולם זמינים (הפתרון של המלצר בבעיית הפילוסופים הסועדים). השחרור לא צריך להיות אטומי.

Priority Inversion היא בעיה שנגרמת במצב שבו תהליך בעדיפות גבוהה חסום ומחכה שמשאב ישמש תהליך בעל עדיפות נמוכה יותר. בנוסף ייתכן מצב שבו התהליך בעלת העדיפות הנמוכה לא יכול לרוץ כי ישנו תהליך נוסף בעל עדיפות ביניים שמקבל זמן CPU.

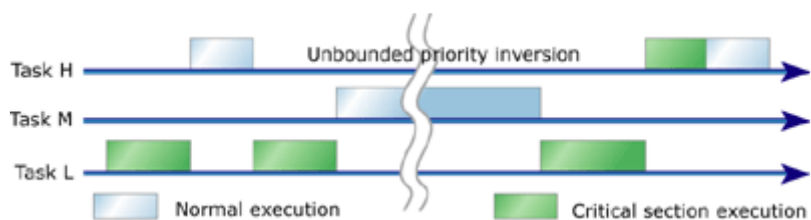
ניתן לחלק את הבעיה ל2 בעיות:

Bounded Priority Inversion הוא מצב שבו יש רק 2 תהליכים (אחד גבוה H ואחד נמוך L) ואז הבעיה חסומה, כלומר התהליך הגבוה יחכה לכל היותר עד שהתהליך הנמוך ישחרר את המשאב.



אם לא היה אפשרות לpreemption לא היה ניתן להגיע למצב כזה כי H לא היה חוסף את המעבד ומחכה בתור של המשאב. כלומר הבעיה נגרמת בעקבות ההתנגשות בין הצורך לpreemption לבין הצורך לבלעדיות ולכן לBounded Priority Inversion אין פתרון.

Unbounded Priority Inversion היא בעיה חמורה יותר אבל אותה כן ניתן לפתור. במצב כזה ישנם יותר מ2 תהליכים: תהליך H, תהליך L ותהליך (אחד או יותר) בעל עדיפות ביניים M. כאשר תהליך L תופס משאב ותהליך H מחכה ש L ישחרר אותו, M יכול לקבל זמן CPU (כיוון שהוא בעל עדיפות גבוהה יותר מ L) ובכך לעקב את H שרדום בתור. בעקבות כך H יכול לפספס את הדדליין שלו.



ההמתנה היא לא חסומה כיוון שיש תהליך נוסף שרץ בין H ל L.

הפתרון הראשון לבעיית הPriority Inversion הוא **Priority inheritance protocol**: התהליך בעל העדיפות הנמוכה ביותר L שהוא הבעלים של המשאב מקבל את העדיפות של התהליך בעל העדיפות הגבוהה ביותר H שביקש את המשאב. בשחרור המשאב L יחזור לעדיפות המקורית שלו. במקרה כזה הבעיה תהפוך לBounded כיוון ששום תהליך בעל עדיפות ביניים M לא יוכל לרוץ כל עוד L הוא הבעלים של המשאב.

הפתרון השני הוא **Ceiling priority protocol**: העדיפות של התהליך שתופס את המשאב מוגדרת לעדיפות הגבוהה ביותר +1 מכל המשימות האפשריות שעשויות לבקש את המשאב מרגע התפיסה ועד השחרור. לפני ההרצה צריך לעבור ידנית על כל התהליכים ולבדוק בין כל התהליכים מה התהליך בעל העדיפות הגבוהה ביותר שעשוי לדרוש את המשאב. העדיפות שלו תהיה ערך הceiling.

הבעיה עם הפתרון הזה הוא שהוא גורם לקיפוח כיוון שלא בטוח שH יבקש בכלל את המשאב בזמן שL תופס אותו אבל בכל זאת H לא יוכל לרוץ. בשונה מקטע אטומי, פתרון זה לא יחסום תהליכים בעלי עדיפות גבוהה יותר מH שלא הולכים לבקש את המשאב.

ניתן לסכם את ההבדלים בין הפתרונות בטבלה:

היווצרות של wait	ערך העדיפות	עיתוי	Priority Inheritance Ceiling Protocol
יכול להיווצר	P_H (העדיפות של H)	כשהמבקש את המשאב	
לא יכול להיווצר	$\text{MAX}\{P_1, P_2, \dots, P_n\}$ כאשר $n=1\dots$ הם התהליכים שעשויים לבקש את המשאב	כשהמבקש את המשאב	

ניתן לשים לב של Ceiling Protocol מונע deadlock וגם priority inversion, בניגוד לPriority Inheritance שמונע רק priority inversion.

בעיה נוספת היא שהPriority Inheritance מחייב שיהיה owner למשאב (כלומר מחייב שימוש במutex) כי צריך לדעת למי להעלות את העדיפות במקרה שH מבקש את המשאב, בניגוד לCeiling Protocol שבו ניתן להשתמש גם בסמפור רגיל כי העדיפות של L תעלה ברגע שL תופס את המשאב. עוד נחזור לנושא הזה בהמשך.

נחזור כעת לדון בסמפורים.

בעת ביצוע Acquire לסמפור ניתן להמתין באחת מהאופציות הבאות:

1. **Wait for ever** – התהליך ירדם בתור עד שיוציאו אותו (אם בכלל)
2. **Wait with a time out** (deadlock rescue) – התהליך ירדם בתור עד שיוציאו אותו או עד פרק זמן מסוים שבו הוא יוותר על המשאב ויצא מהתור. מונע deadlock.
3. **Do not wait** – התהליך כלל לא ייכנס לתור. אם המשאב תפוס התהליך מוותר על השימוש במשאב.

למרות שסמפורים הומצאו בשביל קטעים קריטיים, ניתן לעשות איתם שימושים נוספים לסנכרון. לדוגמא אם יש 2 תהליכים שונים שמחשבים את הנוסחה $c = a + b$ כך שתהליך A מחשב את a ותהליך B מחשב את b ניתן להשתמש בסמפור ע"מ לסנכרן ביניהם:

Task A	Task B
1. some code...	some code...
2. semaphore.acquire()	more code...
3.	b = result
4.	semaphore.release()
5. c = a + b	

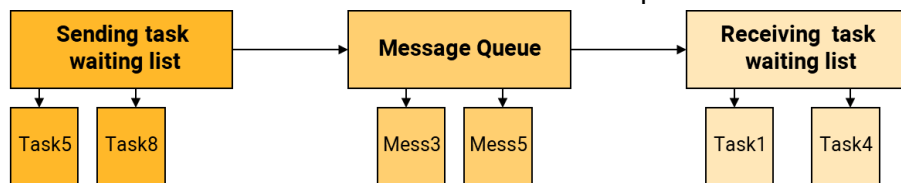
דוגמא נוספת היא הפתרון לבעיית היצרן והצרכן שמשתמש בסמפור מניה ע"מ לפתור את הבעיה. עוד דוגמא היא כאשר תהליך מחכה לאירוע (כמו לחיצה על מקש שמקפיץ את ISR) ויש צורך לשמור את המידע של האירוע אבל לבצע את העיבוד רק לאחר מכן כדי לא לתקוע תהליכים בעלי עדיפות גבוהה יותר. במקרה כזה התהליך ימתין לאיזשהו סמפור נעול, וברגע שהISR יקבל אירוע הוא ישמור את הנתונים וישחרר את הסמפור, כך שהעיבוד יתבצע ע"י התהליך רק בזמן המתאים לו.

אלגוריתמים של סמפורים (כל הפונקציות הבאות חייבות להיות אטומיות):

Type	Acquire	Release
Binary Semaphore	<pre> if (s == 1) s = 0; else { numberSuspended++; waitingList.Put(currentTask); Suspend(currentTask); } </pre>	<pre> if (numberSuspended > 0) { numberSuspended--; waitingList.Get(task); Resume(task); } else s = 1; </pre>

Counting Semaphore	<pre> if (s > 0) s--; else { numberSuspended++; waitingList.Put(currentTask); Suspend(currentTask); } </pre>	<pre> if (numberSuspended > 0) { numberSuspended--; waitingList.Get(task); sResume(task); } else s++; </pre>
Mutex	<pre> if (s == 1 Owner == CurrentTask) s = 0; else { numberSuspended++; waitingList.Put(currentTask); Suspend(currentTask); } Owner = CurrentTask; level++; </pre>	<pre> if (Owner == CurrentTask) if (--level) return; else { Owner = -1; if (numberSuspended > 0) { numberSuspended--; waitingList.Get(task); Resume(task); } else s = 1; } </pre>

Message Queues הוא כלי נוסף לסנכרון בין תהליכים. מטרת Message Queues היא ליצור סנכרון וגם העברת נתונים (בניגוד לסמפורים שמאפשרים רק סנכרון), לייצר התנהגות שדומה לPipelines ולבטל את הצורך של התהליכון לנהל את כל השליחה והקבלה.



כל תהליך שרוצה לשלוח הודעה יכול להפעיל את הפונקציה send. אם יש מקום בתור ההודעות ההודעה תישלח והתהליך יוכל להמשיך לרוץ. אם אין מקום בתור ההודעות השולח ירדם בתור של השולחים עד שיתפנה מקום להודעה שלו ואז היא תיכנס לתור ההודעות והוא יוכל להמשיך לרוץ.

Send(m):

```

if (messageQueue.Full()) {
    senderList.Put(currentTask);
    Suspend(currentTask);
}
messageQueue.Put(m);
if (!receiverList.Empty())
    Resume(receiverList.Get());

```

אותו רעיון בקבלת הודעה: תהליך שרוצה לקרוא הודעה יפעיל את הפונקציה receive. אם יש הודעות בתור ההודעות המקבל יקבל את ההודעה ויכול להמשיך לרוץ. אחרת הוא יירדם בתור של המקבלים עד שתגיע הודעה והוא יתעורר ויקבל את ההודעה.

Receive():

```

if (messageQueue.Empty()) {
    ReceiverList.Put(currentTask);
    Suspend(currentTask);
}
m = messageQueue.Get();
if (!senderList.Empty())
    Resume(senderList.Get());

```

אפשרות נוספת היא שליחה ע"י פונקציית Broadcast (שידור רוחבי): שליחת הודעה לכל מי שנמצא בתור של המקבלים ביחד.

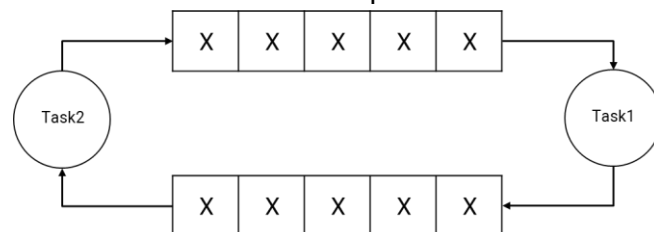
בדרך כלל סדר תור ההודעות הוא FIFO. קיימת אפשרות לשלוח הודעות דחופות (urgent) שעוברות ישר לראש התור, אבל אם כל ההודעות המגיעות דחופות סדר תור ההודעות הופך לLIFO. התורים של השולחים והקוראים יכולים להיות לפי FIFO או לפי עדיפויות. גם פה ניתן להמתין בתור לתמיד, להמתין עד timeout מסוים או לא להמתין בכלל.

Message Queues מבטיח שמי שאין לו מה לעשות לא יבזבז סתם את הזמן של המעבד בהמתנה להודעה שתגיע (busy waiting). זהו עיקרון חשוב בRTOS.

כל הודעה מועתקת פעמיים במשך התהליך. בפעם הראשונה כאשר ההודעה נכנסת לתור ההודעות ובפעם השנייה כאשר ההודעה עוזבת את תור ההודעה (עבור נתונים גדולים ניתן להשתמש במצביעים). כל זה במטרה שההודעה לא תהיה תלויה במי שיצר אותה או במי שמקבל אותה, כלומר מתעסקים עם עותק של ההודעה ולא עם ההודעה עצמה.

דוגמא לשימוש: מצלמה דיגיטלית שמצלמת תמונה. בסיום עיבוד התמונה הISR יכניס את התוצאה לתור וכשיגיע הזמן התהליך שאחראי על התצוגה יוכל להציג אותה לצלם על המסך.

גם בMessage Queues יכול להיווצר מצב של deadlock, לדוגמא במצב הבא שבו 2 תהליכים מחכים כל אחד שיהיה מקום לכתוב בתור אבל התור מלא עד שלא יקראו ממנו:



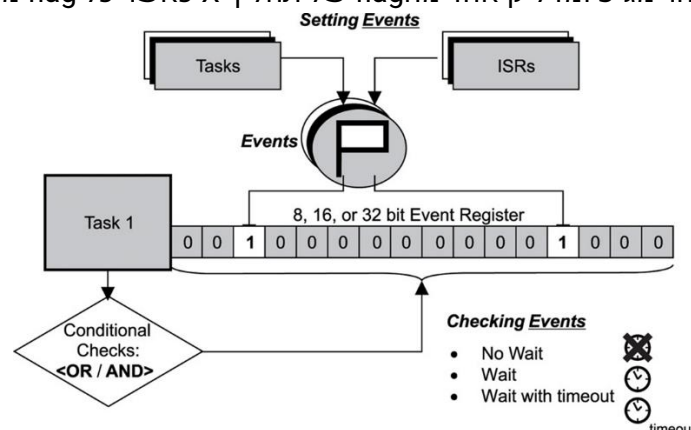
ניתן לפתור את הבעיה בצורה דומה לפתרון של הסמפור עם timeout: התהליך מנסה לשלוח / לקבל הודעה, אבל אם אחרי t יחידות זמן הוא עדיין תקוע של השולחים / מקבלים הוא יוצר מהתור ושובר את השרשרת.

חשוב לשים לב שהISR לא יכול אף פעם להירדם, כיוון שהוא לא תהליך ואין מישהו שיכול להעיר אותו כי הוא נמצא ברמה גבוהה יותר מהסדרן. הסדרן לא מכיר בכלל את הISR, וכשהISR צריך לטפל בפסיקה הוא חוטף לבד את המעבד.

לכן אם ISR רוצה להעביר הודעה לתהליך אבל Message Queues מלא יש בעיה שצריך למצוא לה פתרון בהתאם לסיטואציה. אין פתרון חד משמעי עבור בעיה כזאת. (אותו דבר גם לגבי כל כלי סנכרון אחר שמרדים את התהליך כשאין מקום: הISR לא יכול להשתמש בו בצורה הרגילה).

בFreeRTOS יש לMessage Queue אוסף נפרד של פונקציות עבור ISR ע"מ שלא ישתמש בפונקציות הרגילות.

Event Registers הוא מבנה נוסף לסנכרון שפועל באופן ישיר בין תהליך / ISR לתהליך אחר (בלי מתווך כמו סמפור). לכל תהליך מצמידים מערך של flags (יכול להיות מיוצג ע"י מערך של ביטים) כך שכשרוצים להודיע משהו לתהליך X, תהליך אחר מגיע ומדליק אחד מהflags של תהליך X כאשר כל flag מסמל אירוע אחר.



המתנה לflag או לכמה flags שידלקו נקראת receive. פונקציית receive מקבלת תנאי מסוים כך שהתהליך יתעורר רק כשהתנאי יתקיים.

הדלקה של flag אחד או יותר נקראת send. כל פעם שמתבצע send לתהליך, נבדק גם האם התנאי שלו התהליך מחכה מתקיים. אם הוא מתקיים התהליך יתעורר ובכך למעשה חוסכים את הצורך לבדוק כל CPU burst אם התנאי מתקיים.

המאפיינים של Event Registers הם שהתקשורת לא כוללת נתונים, אין מידע לגבי השולח, אירועים אינם עומדים בתור, אין ספירה של כמה פעמים אירוע התקבל (רק 1 או 0) ויש אפשרות לחכות לנצח או לחכות עם timeout.

על מנת ליישם Event Register במערכת SMARTS יש צורך להוסיף 2 מערכים של ביטים: Flags ו-MaskCondition וכן את 2 הפונקציות הבאות:

```
Send(mask) {
    // bitwise OR: add the bits of value 1 in mask to Flags
    // e.g. Flags = 1110, mask = 0001, Flags | mask = 1111
    Flags |= mask;
    // bitwise AND: Ignore all bits that are not relevant to the mask and check if the mask exists
    // e.g. Flags = 1111, MaskCond = 1001, Flags & MaskCond = 1001 → the condition exists
    If (Flags & MaskCond == MaskCond)
        If (status == Suspend) {
            Resume();
            // bitwise NOT and AND: deleting the bits of MaskCond from Flags
            // e.g. Flags = 1111, MaskCond = 1001, ~MaskCond = 0110,
            // Flags & ~MaskCond = 0110
            Flags &= ~MaskCond;
        }
}

Receive(mask) {
    Maskcond = mask;
    // check if the condition already exists before the process falls asleep
    If (Flags & MaskCond == MaskCond) {
        // deleting the MaskCond from the Flags, similar to send
        Flags &= ~MaskCond;
        Return;
    }
    Else
        Suspend();
}
```

בנוסחת RMA המקורית הייתה מגבלה שעל פיה אסור שתהליכים יקיימו אינטראקציה אחד עם השני. **נוסחת RMA המורחבת** היא הנוסחה לRMA שמורחבת למקרים שבהם כן יש אינטראקציה, והיא רלוונטית רק כשעובדים עם ceiling priority המשופר (מוזכר בהמשך). הנוסחה המורחבת מתבססת על כך שהאפשרויות היחידות לחסום תהליך כלשהו הן או שתהליך (אחד או יותר) בעל עדיפות גבוהה יותר רץ או שהוא חסום ע"י קטע קריטי שמוחזק ע"י תהליך בעל עדיפות נמוכה יותר:

$$\frac{C_1}{T_1} + \dots + \frac{C_i}{T_i} + \frac{B_i}{T_i} \leq i(2^{1/i} - 1)$$

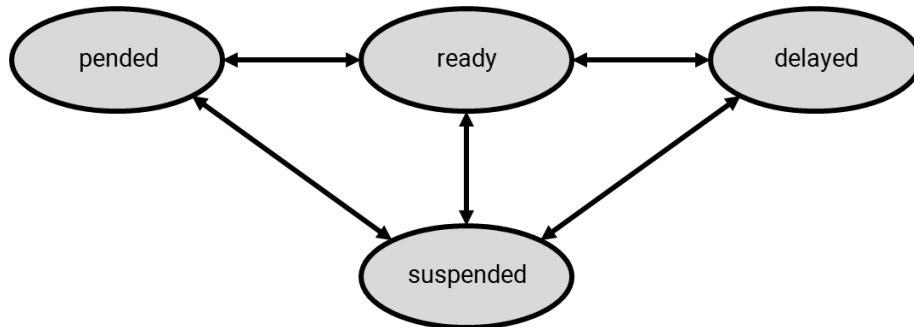
כאשר C הוא זמן הריצה, T הוא זמן המחזור, B הוא זמן החסימה הארוך ביותר, N הוא מספר התהליכים שרצים ותהליכים 1 עד i-1 הם התהליכים בעלי העדיפות שגבוהה יותר מ-i. אם התנאי מתקיים, תהליך i יעמוד בדדליין שלו.

בהמשך מופיעה התראומה ה-12 שמוכיחה שע"י שימוש ש ceiling priority המשופר יכול להיות רק קבוצה אחת בלבד של תהליכים שיכולים לחסום תהליך בהמתנה למשאב כלשהו (כלומר שכולם משתמשים באותו משאב), ולוקחים את משך הקטע הקריטי הכי ארוך מבניהם.

סיכום VxWorks

הליבה של VxWorks, wind, כוללת multi-tasking עם תזמון עדיפות שהוא preemption, סנכרון ותקשורת בין תהליכים, טיפול בפסיקות, טיימרים מסוג watch dog וניהול זיכרון.

תהליך יכול להיות באחד מהמצבים הבאים:



Delayed – התהליך ממתין לtimeout או אירוע דומה אחר.

Pended – התהליך ממתין לתהליך אחר שיעיר אותו (לדוגמה תהליך שרדום בתור של סמפור)

Suspended – התהליך מושע (לא נהוג להשתמש במצב הזה)

בזכות ההפרדה בין delayed לpended קל יותר להבין מתי המערכת נמצאת בdeadlock ומתי לא.

ניתן לחלק את האפשרויות של VxWorks לטיפול בקטע קריטי ל3 רמות:

1. חסימת פסיקות (intLock ו-intUnlock)
2. חסימת הסדרן (taskLock ו-taskUnlock)
3. שימוש בסמפורים

פונקציות שקשורות לסדרן:

פונקציה	הסבר
kernelTimeSlice	שיוני אורך פלח הזמן באלגוריתם RR
taskPrioritySet	שיוני עדיפות של תהליך
taskLock	ביטול / החזרת הקריאות לסדרן, כלומר כל קטע שיהיה בין 2 הפונקציות
taskUnlock	האלו יהיה קטע קריטי

פונקציות שקשורות לסמפורים:

פונקציה	הסבר
semBCreate	יצירת סמפור בינארי (B), מיוטקס (M) או סמפור מנייה (C)
semMCreate	
semCCreate	
semDelete	מחיקת הסמפור ושחרור שלו
semTake	מקביל לacquire
semGive	מקביל לrelease
semFlush	שחרור כל התהליכים בתור אל תוך הקטע הקריטי (לא רלוונטי למיוטקס)

- ביצירת סמפור ניתן להעביר לפונקציה SEM_Q_PRIORITY ואז התהליכים יצאו מהתור ע"פ העדיפות או SEM_Q_FIFO ואז התהליכים יצאו לפי FIFO.
- ביצירת מיוטקס אפשר להעביר לפונקציה את הפרמטר SEM_INVERSION_SAFE שיפעיל Priority Inheritance (מחייב שימוש בתור ע"פ עדיפויות - SEM_Q_PRIORITY).
- ביצירת סמפור אפשר להעביר לפונקציה את הפרמטר SEM_DELETE_SAFE שמונעת מתהליכים אחרים לשלוח kill לבעלים ע"י הפעלה מרומזת של הפונקציה taskSafe בכניסה למיוטקס והפעלת taskUnsafe ביציאה מהמיוטקס.

- סמפור יכול להיות מאותחל ע"י SEM_FULL (1 = פנוי, בניגוד לאיך שזה נשמע) או SEM_EMPTY (0 = נעול).
- הפונקציה semTake יכולה לקבל כפרמטר NO_WAIT, WAIT_FOREVER או מספר חיובי שמבטא ערך timeout במילי שניות.

פונקציות שקשורות לMessage Queue:

פונקציה	הסבר
msgQCreate	יצירת תור הודעות
msgQDelete	מחיקה ושחרור של תור הודעות
msgQSend	שליחת הודעה לתור ההודעות
msgQReceive	קבלת הודעה מתור ההודעות

- כמו בסמפור, ניתן להשתמש בNO_WAIT או WAIT_FOREVER בכניסה לאחד התורים (שולחים או מקבלים).
- בmsgQSend אפשר לציין האם ההודעה היא MSG_PRI_NORMAL (ואז ההודעה תיכנס בסוף התור) או שההודעה היא MSG_PRI_URGENT (ואז היא תיכנס בראש התור).

פונקציות שקשורות לפסיקות:

פונקציה	הסבר
intConnect	מקבלת מספר פסיקה ומצביע לפונקציה ומציבה את המצביע המקום המתאים בווקטור הפסיקות
intContext	מחזירה True אם היא נקראת באמצע פסיקה (שימושי במקרים שבהם יש צורך לבצע משהו שונה עבור טיפול בפסיקה או סתם תהליך רגיל, לדוג' האם להירדם בתור של סמפור כאשר ISR עושה acquire)
intCount	עומק הקינון של פסיקה בתוך פסיקה
intLock	נטרול / אפשרור פסיקות במערכת
intUnlock	

בד"כ RTOS ומערכות Embedded הן מערכות לא מאוישות ואם קורת תקלה צריך להיחלץ ממנה לבד. ע"מ לבדוק שהמערכת רצה בצורה תקינה נהוג שהתהליך בעל העדיפות הכי גבוהה נותן כל הזמן סימני חיים (לדוג' ע"י הפעלה של flag כלשהו). בשביל לבדוק את סימני החיים האלו קיים **Watchdog** שפעם בכמה זמן בודק האם יש סימן חיים. אם אין סימן Watchdog יניח שהייתה שגיאה ויבצע reset למערכת.

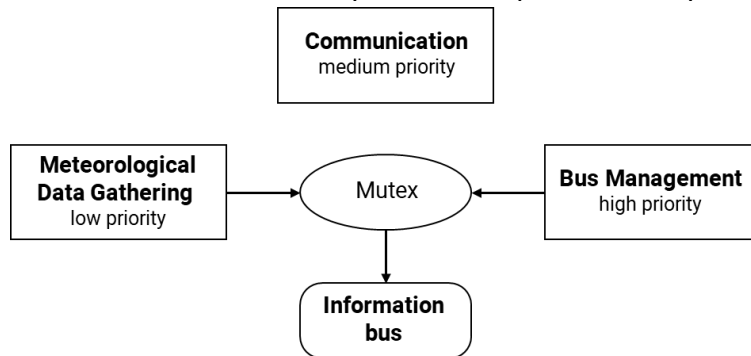
פונקציות שקשורות לWatchdog:

פונקציה	הסבר
wdCreate	יצירה ואתחול של watchdog חדש
wdDelete	עצירה ושחרור של watchdog
wdStart	התחלת הטיימר של watchdog
wdCancel	ביטול המחזור הנוכחי של הטיימר

רכב החלל שנחת במאדים – תיאור התקלה

בשנת 1997 מייק ג'ונס (בכיר במייקרוסופט) היה בכנס של IEEE ושמע הרצאה של מישוהו בשם דוד וילנר שהוא מדען בכיר בחברת Wind River Systems (החברה שייצרה את VxWorks), והוא מספר את הסיפור הבא.

ב-4 ביולי 1997 נחת רכב חלל על מאדים (בשם Mars Pathfinder) שמטרתו היה לאסוף נתונים ותמונות ולשלוח את המידע לכדור"א. כמה ימים אחרי הנחיתה רכב החלל התחיל לבצע reset כל כמה זמן וכל המידע היה מתאפס. רכב החלל רץ על VxWorks וכל המשימות שהיה עליו לבצע חולקו לתהליכים שונים כך שהעדיפות של כל תהליך הייתה בהתאם לרמת הדחיפות של המשימה. ברכב בחלל היה information bus שהיה אחראי על העברת מידע בין התהליכים והגישה אליו הייתה באמצעות מיוטקס. את bus ניהל תהליך בשם bus management, שהיה בעדיפות גבוהה. תהליך נוסף היה תהליך שהיה אוסף מידע מטאורולוגי בעדיפות נמוכה כיוון שהיה רץ לעיתים רחוקות. תהליך זה היה שם את המידע שאסף bus דרך המיוטקס של bus. עוד תהליך היה תהליך של תקשורת שהיה רץ מידי פעם וכן הוא היה בעדיפות בינונית.



אם התהליך הגבוה (נקרא לו H) ניסה לעשות לתפוס את המיוטקס בזמן שהוא היה תפוס ע"י התהליך הנמוך (נקרא לו L), H היה נרדם בתור של המיוטקס עד ש L היה משחרר אותו (בעיית Bounded Priority Inversion טיפוסית). אבל כיוון שהיה תהליך נוסף בעל עדיפות ביניים (M) הבעיה הייתה הופכת לUnbounded. בד"כ המערכת עבדה כמו שצריך, אבל לעיתים רחוקות תהליך M היה נקרא ע"י הסדרן בדיוק כ H היה רדום בתור של המיוטקס. כיוון ש M היה בעל עדיפות גבוהה יותר מ L ואורך הריצה שלו היה ארוך מאוד יחסית, אזי היה נוצר מצב שבו M רץ במשך זמן ארוך בזמן ש L מחכה שיסיים ולמעשה H נשאר רדום בתור. אחרי כמה זמן היה עולה watchdog שהיה מזהה ש H לא רץ במשך הרבה זמן והיה עושה reset לכל המערכת.

איך עלו על הבעיה?

בVxWorks קיימת אופציה לכתוב כל מה שקורה לתוך logger, כולל ביצוע דברים בסיסיים כמו context switch וכו'. מהנדסי הפרויקט הריצו על יבש את כל התרחשים שקרו במערכת כפי שדווח log וניסו שעות ארוכות להבין מה גורם לבעיה. בסופו של דבר הצליחו המהנדסים להבין שהreset נובע בגלל priority inversion.

איך הבעיה תוקנה?

כידוע בVxWorks יש אפשרות מובנת לpriority inheritance אולם במקור האפשרות הזאת לא הייתה מופעלת ברכב החלל. למזלם הייתה להם אופציה לעשות עדכון גרסה ולשלוח את הגרסה המעודכנת מכדור"א לרכב החלל, לכן הם יכלו להפעיל את האופציה הנ"ל ובכך הבעיה טופלה.

ניתוח ומסקנות

ראשית, אם לא היה logger לא היו עולים לעולם על הבעיה. לכן חשוב בRTOS (ובכל מערכת בכלל) לבצע log לכל פעולה שקורה. שנית, חשוב שיהיה אפשרות לבצע שינויים גם מרחוק, אחר לא היה ניתן לטפל בבעיה.

לבסוף, ההנחה של המהנדסים במקור ש"התהליך bus management מבוצע בתדירות גבוהה מאוד והוא קריטי בזמן - אנחנו לא צריכים להשקיע בו את הזמן הנוסף לביצוע ירושה בעדיפות" הייתה שגויה לחלוטין. כלומר, לא מעלים ביצועים על חשבון אמינות.

דוד ווילנר סיפר שמהנדסי התוכנה ראו שהבעיה של resetn קרתה פעמים בודדות עוד לפני השיגור למאדים אבל כיוון שלא זיהו את מקור הבעיה הם הניחו שזה בעיה בחומרה ולא בעיה בתוכנה. כלומר חשוב לנסות להבין בעיות עד הסוף ולא לנסות להשליך את הבעיות על צוותים אחרים.

ניתן ללמוד מהסיפור הזה גם את החשיבות של אנשי האלגוריתמיקה והתיאוריה. אם לא היו מפרסמים את המאמרים על ירושות עדיפויות לא היו עולים על הבעיה ובטח שלא היו מגיעים לפתרון שלה בתוך רכב החלל.

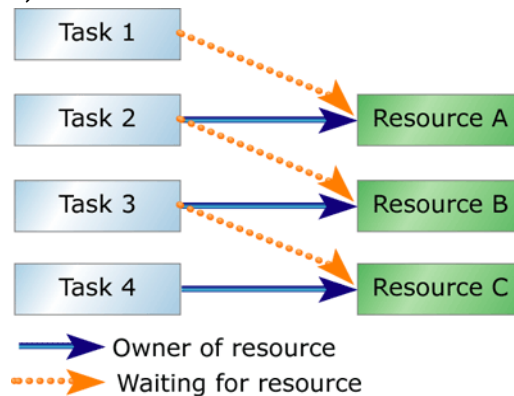
מאמר בעד Priority Inheritance (How to use priority inheritance)

כותב המאמר מזכיר: טעויות קטלניות, deadlock ובאגים לא ברורים מחכים למתכנת שלא מקפיד על היפוך עדיפות.

שיתוף משאבים במערכת שהיא preemption יכול ליצור התנגשויות, כמו deadlock וpriority inversion. דוגמה מוכרת לבעיה שנוצרה עקב priority inversion היא רכב החלל של מאדים בשנת 1997 שכמעט כשל לגמרי בגלל הבעיה. אם היו לוקחים בחשבון מראש שיכול להיות priority inversion לא היו נתקלים כלל בבעיה. במאמר הנ"ל נדון בדרכים לפתור priority inversion אבל **עדיף לחמוק מהבעיה באופן גורף מאשר להתמודד איתה.**

כותב המאמר מסביר על ההבדל בין bounded priority inversion לunbounded priority inversion, אין צורך לחזור על זה שוב.

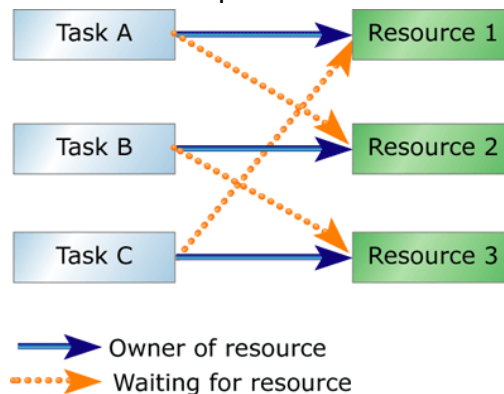
בנוסף, הכותב מציג בעיה נוספת: **שרשרת מקוננת של קטעים קריטיים** היא כאשר יש שרשרת של hold & wait כך שתהליך העליון בתור הוא בעל העדיפות הגבוהה ביותר והוא נאלץ לחכות לסיום הריצה של כל התהליכים שמתחתיו, כאשר התהליכים תפסו את המשאבים לפי סדר העדיפות שלהם (1 הכי גבוה, 4 הכי נמוך).



נוצר מצב ש Task 1 צריך להמתין במשך סכום כל זמני הריצה של שאר התהליכים. למה סכום, אם למעשה הקטע קריטי שרלוונטי ל B גם אצל Task 2 וגם Task 3, וקטע קריטי של C גם אצל Task 3 וגם אצל Task 4? למעשה Task 1 צריך להמתין ל $A_2 + B_2 + B_3 + C_3 + C_4 = A + 2B + 2C$! אנחנו דנים רק בקטעים קריטיים מקוננים ("בבושקות") במאמר הזה, כלומר אם Task 3 תפס את B ואז את C, הוא ישחרר קודם את C ואז את B ולכן בכל מקרה זמן השימוש ב C מוכל בזמן השימוש ב B ולכן ניתן לחשב את זמן ההמתנה של Task 1 כסכום של כל הקטעים הקריטיים שמתחתיו. עבור קטעים קריטיים מקוננים אין פתרונות כאשר נוצרת בעיית priority inversion.



אם התהליך האחרון בשרשור יבקש גם את המשאב הראשון ייווצר deadlock:



פתרון ראשון: Priority ceiling protocol

זהו הפתרון הראשון שהומצא מבחינה כרונולוגית.

בזמן שתהליך תופס משאב הוא מקבל את עדיפות גבוהה יותר ב-1 מהעדיפות של התהליך עם העדיפות הכי גבוהה שעשוי לדרוש את המשאב. כך מבטיחים שאף תהליך שירצה את המשאב לא יוכל לעשות preemption בזמן שהמשאב תפוס ולהירדם בתור.

החיסרון המרכזי של השיטה הזאת היא הצורך במעבר ידני על כל הקוד וחיפוש העדיפות הכי גבוהה שעשויה לדרוש את המשאב. יכול להיות גם בלתי אפשרי לחישוב באפליקציות מורכבות מידי.

עוד חיסרון התקורה במקרה הממוצע גבוהה יותר כי הקפצת העדיפות קורת כל פעם שהתהליך תופס את המשאב (לעומת priority inheritance שמעלה עדיפות רק אם יש צורך בזה).

מצד שני, בשיטה הזאת אין שום wait בגלל שאם תהליך תופס משאב הוא רץ בעדיפות גבוהה יותר מכל האחרים שעשויים לרצות את המשאב הזה ולכן הוא ישחרר את המשאב ורק אז תהליך אחר יוכל לבקש אותו. לכן גם לא יכול להיות deadlock.

שיטה זו גם יוצרת בעיה קשה של קיפוח – התהליך שתופס את המשאב ירוץ בעדיפות גבוהה יותר מכל אלו שמתחננו גם אם אין בכך צורך כלל ולכן יתפוס זמן CPU על חשבון תהליכים אחרים.

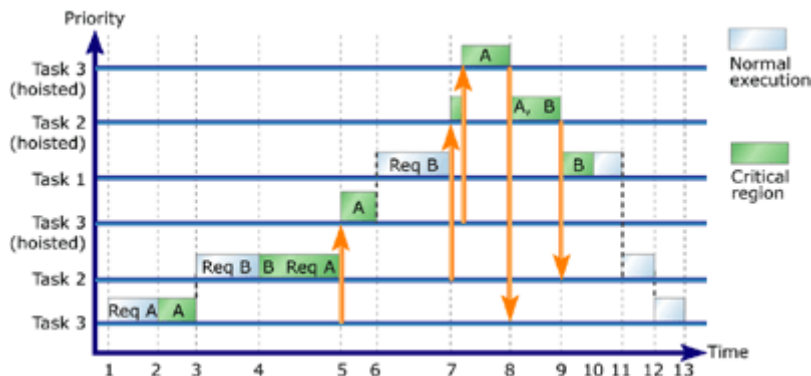
פתרון שני: Priority inheritance protocol

בשיטה זו מעלים את העדיפות של הבעלים של משאב רק אם תהליך אחר בעל עדיפות גבוהה יותר מהבעלים מבקש את אותו משאב. כשהבעלים משחרר את המשאב הוא חוזר לעדיפות המקורית שלו. התקורה הממוצעת בשיטה זו היא נמוכה כיוון שהעלאת העדיפות של הבעלים תקרה רק אם תהליך בעל עדיפות גבוהה יותר יבקש את המשאב.

במקרי הקצה ceiling יותר טוב כי אין בו שרשור, ולכן inheritance יכול להיות deadlock.

אם משתמשים ב-Priority inheritance ובמקביל מחליטים שכל תהליך יכול להחזיק רק משאב אחד בו זמנית לא יכול להיות hold & wait אז ימנע מצב של deadlock כמו ב-ceiling protocol.

ההורשה יכולה להסתבך כשאר יש שרשור, כיוון שהמשאב הגבוהה צריך "לחלחל" את העדיפות שלו לאורך כל השרשור. מקרה לדוגמה:



לצורך כך מערכת ההפעלה תצטרך להחזיק רשימה של כל המיוטקסים במערכת. ברגע שתהליך H מוריש את העדיפות שלו לתהליך M, המערכת תצטרך לעבור על כל המיוטקסים ש-M נמצא בהם ולעדכן בהם את העדיפות, ואז לעבור על כל מי שהיא עדכנה בהם את העדיפות ולהמשיך לעדכן בצורה רקורסיבית על כל המיוטקסים.

בעיה זו מסבכת את המימוש של priority inheritance...

סיכום היתרונות והחסרונות של 2 השיטות (אדום – חיסרון, כחול – ייתרון):

Priority inheritance	Ceiling protocol	
אין צורך	יש צורך	מעבר ידני
יכול להיות	לא יכול להיות	שרשור של קינון של קטעים קריטיים
יכול להיות	לא יכול להיות	deadlock
נמוכה	גבוהה	תקורה ממוצעת
אין	יש	קיפוח בינוניים

- כותב המאמר ממליץ באופן כללי להימנע מכל מצב של priority inversion ומביא כמה דרכים להתחמק מהבעיה:
1. הימנעות מוחלטת משיתוף משאבים במידת האפשר. אם אי אפשר כדאי לשקול לאחד את 2 התהליכים לתהליך אחד משותף.
 2. לתת לכל התהליכים שניגשים לתהליך משותף את אותה עדיפות (רק במקרה שאין שימוש בRR). יפתור את הwait.
 3. **Gate keeper / Server Task** – גישה למשאב תעשה ע"י תהליך אחד שאחראי על המשאב. כל תהליך שרוצה לגשת למשאב פונה אל הServer Task והוא מטפל בבקשה שלו. אם הServer תפוס התהליך המבקש יירדם אבל בכל מקרה לא נוצרת בעיה של priority inversion כי אין hold אלא רק wait. הServer צריך להיות בעל עדיפות גבוהה יותר מכל התהליכים שעשויים לדרוש את המשאב (מצריך מעבר ידני על הקוד וגם גורם לקיפוח כי הServer יכול לטפל בבקשות של תהליכים נמוכים מייד כשהם מבקשים את המשאב).
- לסיכום עדיף להימנע מpriority inversion מאשר לטפל בה. אם אין ברירה עדיף להפוך את הבעיה לbounded מאשר להשאיר אותה unbounded.
- אף אחת מהשיטות לטיפול בpriority inversion לא מושלמת וצריך לבחור את השיטה שיותר מתאימה לאפליקציה הספציפית שאותה מפתחים.

מאמר נגד Priority Inheritance (Against priority inheritance)

הכותב מביא פתיח דומה לפתיח במאמר הקודם ובו הוא מסביר על priority inversion ועל הפתרונות השונים.

RTLinux לא אפשרו priority inheritance מהסיבה הפשוטה שהוא לא אמין, לא יעיל ודורש מימוש מסובך מדי. priority inheritance סובל מחסרונות רבים שלא ניתן לטפל בהם ולכן לא כדאי להשתמש בו בכלל. באופן כללי כדאי להימנע לחלוטין מpriority inversion.

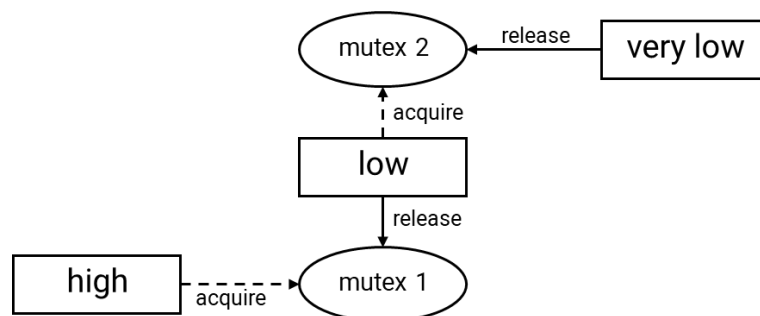
בעיה 2.1: קינון של קטעים קריטיים

כאשר יש שרשרת של קטעים קריטיים יש צורך להוריש את העדיפות של התהליך הגבוה בשרשרת לכל שאר החוליות בשרשרת (טרנזיטיביות). טרנזיטיביות זו גורמת להרבה תקורה בזמן החישוב כי היא דורשת חישובים רקורסיביים ארוכים. זמן ההמתנה של התהליך הגבוה בשרשרת הוא סכום של כל הקטעים הקריטיים שנמצאים תחתיו.

עוד בעיה שיכולה להיות היא שתהיה באפליקציה א' שרשרת ובאפליקציה ב' תהיה שרשרת נוספת שלא קשורה לזו של אפליקציה א' אבל שניהם יהיו מחוברות באמצעות מיוטקס עלום שם של מערכת ההפעלה. כלומר 2 שרשראות שמחוברות ביניהם בלי שבכלל יהיה אפשרות לדעת שהן מחוברות. בעיה נוספת היא ניתוק של שרשרת: אם תהליך באמצע השרשרת עושה acquire עם timeout ובסיום timeout הוא ינתק את השרשרת, יהיה צורך לבצע המון חישובים נוספים ע"מ להחזיר את העדיפות המקסימלית של כל התהליכים שהיו תחתיו בשרשרת.

בעיה 2.2: פעולות מעורבות בירושה וללא ירושה

Priority inheritance יוצא מנקודת הנחה שהשחרור והתפיסה נעשית ע"י אותו מיוטקס, וגם שיש שמירה על הבעלות (כלומר יש owner למיוטקס) ולכן ניתן להוריש עדיפויות. אבל לפעמים השחרור והתפיסה נעשים בצורה נפרדת זו מזו, ולפעמים לא ניתן לשמור owner למיוטקס וצריך להשתמש בסמפור. נניח שיש איזשהו Pipe שניתן גם לכתוב וגם לקרוא ממנו. נוצרת בעיה כיוון שיש לו 2 כניסות: מיוטקס אחד לקוראים ומיוטקס אחד לכותבים. איך יוכל תהליך שמחכה בתור של הקוראים להוריש את העדיפות שלו גם לתהליכים שנמצאים במיוטקס של הכותבים, אם זה בכלל מיוטקס אחר לחלוטין? דוגמה נוספת היא המקרה הבא:



low ישחרר את mutex 1 רק כשהוא יקבל את mutex 2 שתפוס ע"י very low, אבל very low לא יכול לרוץ כי תהליכים בעלי עדיפות ביניים עוקפים אותו וhigh לא יכול להוריש לו את העדיפות כי הוא לא מודע אליו בכלל. במקרה של יצרן וצרכן אין אפשרות להוריש עדיפויות כי היצרן שתפס ראשון את המיוטקס הוא לא האחד שישחרר אותו, וכנ"ל לצרכנים. האם יהיה צורך להוריש את העדיפות לכל מי שיכול לשחרר את המשאב? לפעמים גם יכול להיות מצב שבו לא יודעים מי ישחרר את המשאב, כמו בבעיית הקוראים-כותבים. יש יותר מקורא / כותב אחד שיכול לשחרר את המשאב. אם הקורא האחרון שיוצא מהקטע קריטי משחרר את המשאב אז למי צריך להוריש את העדיפות, אם אין אפשרות לדעת איזה קורא הולך לשחרר את המשאב? במקרה כזה עדיף להשתמש בceiling בגלל שהוא מצריך מעבר ידני על הקוד ואז ככל הנראה יתגלו בעיות כאלו.

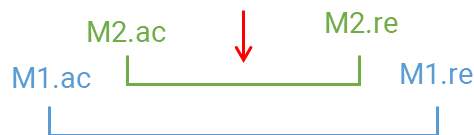
בעיה 2.3: ביצועים

- כל הורשה גורמת להרבה מאוד חישובים:
1. חסימת התהליך בעל העדיפות הגבוהה
 2. העברת העדיפות למי שצריך
 3. שיחזור העדיפות בעת ביצוע release וכו'.

אם הקטעים הקריטיים הם קצרים כדאי פשוט להפוך כל קטע קריטי לקטע אטומי. אם נהפוך כל קטע קריטי לקטע אטומי אז ההמתנה הכי ארוכה תהיה הקטע קריטי הכי ארוך של לא משנה איזה תהליך. פתרון זה יחסוך את כל העיקוב בביצועים, ימנע priority inversion וdeadlock אבל יגרום לקיפוח. ירושה תהיה עדיפה על הפיכה לקטע אטומי רק במצב שבו הסכום של כל הקטעים הקריטיים (כולל כל התקורה) בשרשור יותר קטן מהקטע קריטי הכי ארוך של משאב שלא בקבוצה של השרשור.

בעיה 2.4: ביצועים של מערכת ההפעלה

נניח שתהליך L כלשהו תפס 2 מיוטקסים בצורה מקוננת, ובזמן שהוא היה בעלים על המיוטקס M2 תהליך H בעל עדיפות גבוהה יותר מL עשה acquire למיוטקס M1 (החץ האדום מבטא את הזמן שבו H ביצע ניסה לתפוס את המיוטקס M1):



כשאר L רוצה לעשות release למ2 הוא לא יכול לחזור לעדיפות האחרונה שהייתה לו לפני שהוא תפס את M2 כי אז הוא לא יחזור לעדיפות החדשה שH העניק לו. כלומר לא ניתן לעבוד בצורה של מחסנית, אלא צריך לשמור רשימה של כל המיוטקסים של L הוא הבעלים שלהם, וכל פעם שL עושה release לעבור על כל המיוטקסים ולראות למי מבניהם יש את העדיפות הכי גבוהה ולשנות את העדיפות של L אליה. כל החישובים האלה מוסיפים הרבה תקורה למערכת ההפעלה.

במקור המתכננים של VxWorks ניסו להתגבר על הבעיה בכך שהם הורידו את העדיפות רק כתהליך שחרר את המשאב האחרון שאותו הוא מחזיק. כלומר תהליך יכול לרשת עדיפויות של תהליכים אחרים, אבל הוא יחזור לעדיפות המקורית שלו רק כשהוא יעשה release למשאב האחרון שהוא מחזיק. יוצר קיפוח. עוד תקורה שנוספת היא שכאשר עדיפות של תהליך משתנה צריך למיין שוב את כל התורים של כל המיוטקסים שבהם הוא נמצא ע"מ שיהיו מעודכנים בסדר החדש. (טענה זו נתונה לוויכוח כי ניתן למיין את התור רק כשרוצים למצוא את המקסימלי בו, אין צורך לדאוג שהוא תמיד יהיה ממויין).

בסיום המאמר כותב המאמר אומר שאין תחליף לניתוח ידני של הקוד ונותן הכותב 3 עצות:

1. להפוך קטעים קריטיים קצרים לאטומיים.
2. לנסות להוריד את התחרות על המשאבים (לדוגמה ע"י flip buffer – 2 buffers שונים שפעם הכותב כותב לאחד והקורא קורא מהשני ואז מחליפים ביניהם את buffers).
3. שימוש בServer Task כמו שהציע הכותב של המאמר הקודם.

סיכום הטענות ע"פ ד"ר טייטלבוים:

בעיה 2.1:

- א. המתנה ארוכה לסכום ק"ק
- ב. חיבור שירשורים ע"י מיוטקס עלום
- ג. ניתוק שרשרת

בעיה 2.2:

- א. לא לכל כלי סנכרון יש OWNER
- ב. זה בנוי רק אם הנעילה והשחרור נעשים ע"י אותו תהליך.
- ג. בדוגמה של היצרן והצרכן יש כמה "משחררים" פוטנציאליים.
- ד. בדוגמה של הקוראים והכותבים יש בו זמנית כמה משחררים.
- ה. ואם תטען הרי הבעיה קיימת גם ב CEILING, התשובה תהיה שהניתוח הידני מקיף את הבעיה, או לפחות תגלה אותה.

בעיה 2.4:

א. שחזור העדיפות מסובכת.

ב. כל הורשה דורשת מיון של כל

התורים של כל מיוטקס שבשרשרת.

ג. ההצעה של VxWorks לפתרון א היא

לא יעילה

מאמר הceiling המשופר

(Rate Monotonic Analysis for Real-Time Systems)

מאמר זה מציע פתרון משופר לבעיית priority inversion ע"י שילוב של 2 הפתרונות הקודמים. פתרון זה מורכב מ3 כללים.

כלל 1: תהליך רוצה משאב תפוס

אם תהליך מנסה לתפוס משאב תפוס הוא מוריש לבעלים של המשאב את העדיפות שלו (אם יש בכך צורך). כלומר מבצעים priority inheritance רגיל.

כלל 2: תהליך רוצה משאב פנוי

אחד מהחסרונות הבולטים של ceiling protocol הוא הקיפוח, והceiling המשופר רוצה לנסות לפתור את הקיפוח ע"י כך שהוא יצור מצב שיהיה hold, יהיה wait אבל לעולם לא יהיה hold & wait. לפני ההרצה צריך לחשב לכל משאב את ערך הceiling שלו. הערך הזה לא בא לידי שימוש אלא הוא רק מהווה סוג של תמרור אזהרה. כל פעם שתהליך רוצה לתפוס משאב פנוי, כלומר להיות holder, הוא חייב קודם כל לבדוק שהוא לא הולך להיות waiter ע"י כך שהוא בודק שכל המשאבים שהוא הולך לבקש יהיו פנויים – אם הceiling של כל שאר המשאבים התפוסים בתוכנית קטנים מהעדיפות של התהליך הנוכחי סימן שהוא לא הולך לחכות לאף משאב. במילים אחרות, תהליך לעולם לא יהיה מעוניין במשאב בעל עדיפות קטנה משלו, כיוון שאם הוא היה מעוניין בו העדיפות של המשאב הייתה גבוהה משל התהליך. לכן אם כל המשאבים התפוסים בעלי עדיפות קטנה משל התהליך הנוכחי הוא לא יהיה waiter. הceiling עצמו לא נורש לאף תהליך כיוון שהירושה היא לפי priority inheritance, אבל בכל זאת השיטה הזאת מורידה באופן משמעותי את הקיפוח שקיים בceiling protocol. אם התנאי לא מתקיים, כלומר קיים משאב תפוס בעל עדיפות גבוהה מזו של התהליך הנוכחי, התהליך הנוכחי יירדם בתור של אותו משאב (כי הוא זה שמונע ממנו לרוץ). אם יש כמה כאלה נרדמים באחד הכי גבוהה.

כלל 3: תהליך לא רוצה שום משאב

אם תהליך לא רוצה שום משאב הוא רץ כרגיל, בלי שום ceiling כאשר סדר התהליכים נקבע ע"י הסדרן ע"פ העדיפויות של התהליכים.

השיטה הזאת מבטלת את hold & wait ולכן היא מבטלת מצב של שרשור ולכן גם deadlock, מבטלת את הקיפוח (כמעט) אבל לא פותרת את הבעיה של מעבר ידני על הקוד.

הנוסחה המורחבת של RMA הומצאה בעקבות הceiling המשופר, והיא טוענת שצריך לחכות לסכום כל הריצות של תהליכים בעלי עדיפות גבוהה יותר + הקטע הקריטי הכי גדול מבין התהליכים שקטנים מהתהליך שרץ. הקטע הקריטי היחיד שיכול לעכב את התהליך הוא קטע קריטי של תהליך בעל עדיפות גבוהה יותר, כיוון שקטעים קריטיים של תהליכים בעלי עדיפות נמוכה יותר לא יכולים לרוץ בזמן שתהליך שגבוה מהם מחכה ע"פ שיטה זו.

המאמר המקורי של ceiling הבסיסי

(Experience with Processes and Monitors in Mesa)

המאמר הזה פורסם בפברואר 1980, עוד לפני שבכלל הגדירו את priority inversion בשם שלו. המאמר מתאר את priority inversion באופן הבא:

ביישומים מסוימים רוצים להשתמש בדיסציפלינת תזמון ע"פ עדיפות להקצאת המעבד/ים לתהליכים שאינם ממתנים. אם לא נזהרים, ניתן לפגוע בסדר הנובע מסדר העדיפויות על ידי המוניטורים.

נניח שיש שלוש רמות עדיפות (3 הגבוהות ביותר, 1 הנמוכה ביותר) ושלושה תהליכים P_1, P_2 ו- P_3 , בעלי עדיפויות 1-3 בהתאמה. נניח ש- P_1 ו- P_3 מתקשרים באמצעות מוניטור M ונניח שהתרחש רצף האירועים הבא:

1. P_1 נכנס ל-M.

2. P_2 חוטף את המעבד ל- P_1 .

3. P_3 חוטף את המעבד ל- P_2 .

4. P_3 מנסה להיכנס למוניטור, ונרדם בתור.

5. P_2 פועל שוב ויכול למנוע מ- P_3 לפעול בניגוד לסדרי עדיפויות.

דרך פשוטה להימנע ממצב זה היא לשייך לכל מוניטור את העדיפות הגבוהה ביותר של תהליך שיכנס אי פעם למוניטור זה. ואז בכל פעם שתהליך נכנס למוניטור, העדיפות שלו עולה באופן זמני לעדיפויות של המוניטור.

המאמר המקורי של הורשה ושל ceiling המשופר

(Priority Inheritance Protocols)

מאמר זה פורסם בספטמבר 1990 ע"י IEEE והוא מביא לראשונה את Priority Inheritance ואת ceiling המשופר. המאמר מלא בהוכחות מתמטיות שונות. בתחילת המאמר מוסבר מה זה unbounded priority inversion ולמה זה בעייתי בRTOS.

המאמר מציג בפעם הראשונה את הרעיון של Priority Inheritance (מלווה בכל מיני הוכחות מתמטיות שלא רלוונטיות לקורס):

כאשר תהליך J חוסם תהליך אחד או יותר בעלי עדיפות גבוהה יותר, J מתעלם מהעדיפות המקורית שלו ומבצע את הקטע קריטי בעדיפות הכי גבוהה מבין כל העדיפויות שהוא חוסם. בסיום הקטע הקריטי J חוזר חזרה לעדיפות המקורית שלו.

לאחר מכן המאמר מציג את Ceiling Protocol המשופר:

המטרה של פרוטוקול זה היא למנוע היווצרות של deadlock ושל שרשרת חסימות.

בגדול צורת הפעולה של הפרוטוקול הזה היא לוודא שכשאר תהליך J חוטף את המעבד לקטע קריטי של תהליך אחר ומריץ את הקטע קריטי שלו Z, העדיפות של Z בוודאות תהיה גדולה יותר מכל שאר הקטעים הקריטיים שJ עקף אותם בתור למעבד. אם התנאי הזה לא מתקיים, J לא יוכל להיכנס לZ ונרדם בתור של התהליך שחוסם את J. התהליך שחוסם את J יורש את העדיפות שלו.

הרעיון הזה מחייב השמה של ערך ceiling לכל סמפור, כך שהערך הזה הוא העדיפות הגבוהה ביותר של תהליך שעשוי לבקש את הסמפור.

בנוסף המאמר מציג את התראומה הבאה.

התראומה 12:

נוכיח בשלילה שבceiling המשופר לא יכול להיות יותר מקטע קריטי אחד שחוסם משאב.

יהי J_1, J_2 תהליכים כך שהעדיפויות שלהם P_{J_1}, P_{J_2} מקיימות את היחס הבא: $P_J > P_{J_1} > P_{J_2}$.

יהי 2 מיוטקסים M_1 ו M_2 כך ש M_1 תפוס ע"י J_1 ו M_2 תפוס ע"י J_2 ונניח בשלילה שגם M_1 וגם M_2 מעכבים את J. נסמן ב CP_1 ו CP_2 את ערך ceiling של M_1 ו M_2 בהתאמה.

כיוון ש J_1 תפס את M_1 אזי בהכרח חייב להתקיים $P_{J_1} > CP_2$ (אחרת לא יכול היה להיות מצב שגם M_1 וגם M_2 תפוסים).

אם CP_2 מעכב את J זה אומר שמתקיים $P_J \leq CP_2$, כלומר מתקיים $P_J \leq CP_2 < P_{J_1}$ וזה בסתירה למה שהנחנו בהתחלה ש $P_J > P_{J_1}$!

נובע מכך שלא יכול להיות יותר מקטע קריטי אחד שמעכב את תהליך. מש"ל.

הפטריוט – תיאור התקלה (Patriot Missile Defense)

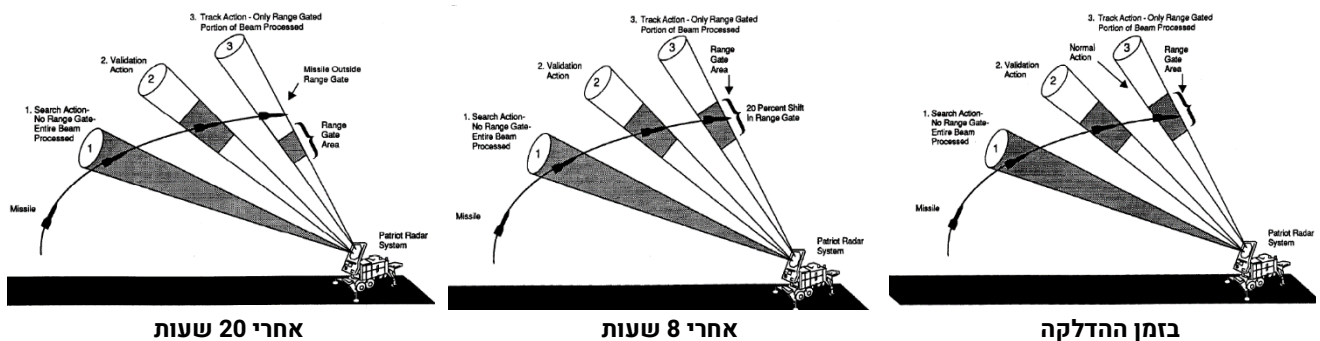
תיאור מקרה שבו בעיית תוכנה הובילה לבעיית מערכת שפגעה בחיי אדם.

הפטריוט היא מערכת הגנה נידת נגד טילים בטווח קצר שפותחה בשנות ה-60. במקור הפטריוט עוצב באירופה על מנת להגן מפני טילים סובייטים. כדי שלא יזהו את המיקום של הפטריוט היו מעבירים אותו פעם בכמה שעות למקום אחר.

המחשב עליו פעל הפטריוט היה מחשב משנות ה-70 שהיה די פרימיטיבי ולכן לא היה טוב כ"כ בביצוע חישובים מדויקים מאוד.

ב-25 בפברואר 1991 מערכת ההגנה "פטריוט" שפעלה בעיר דהארן שבערב הסעודית במהלך מבצע Desert Storm נכשלה בזיהוי וחיסול של טיל סקאד. אותו סקאד נפל והרג 28 חיילים אמריקאים.

קצת קודם לכן, ב-11 בפברואר 1991, המשרד שאחראי על הפטריוט קיבל מידע ישראלי שדיווח שטווח הזיהוי של הפטריוט זר ב-20% לאחר 8 שעות פעולה. אחרי 20 שעות הפטריוט בכלל לא מזהה טילים כי טווח הזיהוי שלו זר לחלוטין.



הפתרון הפשוט ביותר לבעיה יכל להיות פשוט לעשות reset למערכת כל כמה שעות. תהליך כזה היה לוקח בין 60 ל-90 שניות. ככה פעלו בארץ.

המפתחים של הפטריוט החליטו שהשימוש הישראלי בפטריוט הוא שימוש לא טיפוסי והם הניחו שאף אחד אחר לא משאיר את המערכת דולקת ליותר מ-8 שעות. בעקבות ההנחות האלו ב-16 בפברואר 1991 יצאה גרסה מתוקנת עם טיפול חלקי בבעיה, אבל בתיקון הם לא התחשבו במשך הזמן שהמידע הישראלי הזהיר לגביו. ב-21 בפברואר נשלחה הודעה לכל הבסיסים הצבאיים שיכולות להיות בעיות במערכת בעקבות הפעלה ארוכה, אבל בהודעה לא נאמר כמה זמן נחשב "הפעלה ארוכה". בנוסף נאמר בהודעה שקיים עדכון לגרסה והוא בדרך. ב-25 בפברואר בבסיס דהארן הייתה סוללה של פטריוט שפעלה למעלה מ-100 שעות ברציפות, כלומר המערכת כבר מזמן איבדה את טווח הזיהוי שלה. באותו יום נפל טיל סקאד על הבסיס והרג 28 חיילים בו. יום למחרת, ב-26 בפברואר הגיע לבסיס העדכון לתוכנה...

הבעיה עצמה היתה שבשביל לחזות את מיקום נפילת הטיל היו במערכת מספרים ממשיים שייצגו את המהירות ואת הזמן. בגלל שמדובר היה במחשב מיושן שהכיל רק 24 ביטים, התרגום של הזמן Integer למספר ממשי היה יכול להיות רק בטווח של 24 ביטים ולכן נחתך מידע. הייצוג של הזמן היה בעשירות שניה, ואי אפשר להציג עשירית בצורה מדויקת ב-Floating Point בגודל 24 ביטים.

המסר מהסיפור הזה הוא שמהנדסים צריכים לקחת אחריות על מה שהם עושים, וצריך לדייק בכל דבר עד הסוף. אי אפשר לשחרר מוצר בלי לבדוק אותו כמה פעמים לפני ובלי להיות בטוחים שהוא עובד בצורה מדויקת, בטח לא כשמדובר במוצר שחיי אדם תלויים בו.